A Container Security Survey: Exploits, Attacks, and Defenses

OMAR JARKAS, The University of Queensland, Brisbane, Australia
RYAN KO, The University of Queensland, Saint Lucia, Australia
NAIPENG DONG, The University of Queensland, Saint Lucia, Australia
REDOWAN MAHMUD, Curtin University, Perth, Australia

Containerization significantly boosts cloud computing efficiency by reducing resource consumption, enhancing scalability, and simplifying orchestration. Yet, these same features introduce notable security vulnerabilities due to the shared Linux kernel and reduced isolation compared to traditional virtual machines (VMs). This architecture, while resource-efficient, increases susceptibility to software vulnerabilities, exposing containers to potential breaches; a single kernel vulnerability could compromise all containers on the same host. Existing academic research on container security is often theoretical and lacks empirical data on the nature of attacks, exploits, and vulnerabilities. Studies that do look at vulnerabilities often focus on specific types. This lack of detailed data and breadth hampers the development of effective mitigation strategies and restricts insights into the inherent weaknesses of containers. To address these gaps, our study introduces a novel taxonomy integrating academic knowledge with industry insights and real-world vulnerabilities, creating a comprehensive and actionable framework for container security. We analyzed over 200 container-related vulnerabilities, categorizing them into 47 exploit types across 11 distinct attack vectors. This taxonomy not only advances theoretical understanding but also facilitates the identification of vulnerabilities and the implementation of effective mitigation strategies in containerized environments. Our approach enhances the resilience of these environments by mapping vulnerabilities to their corresponding exploits and mitigation strategies, especially in complex, multi-tenant cloud settings. By providing actionable insights, our taxonomy helps practitioners enhance container security. Our findings have identified critical areas for further investigation, thereby laying a comprehensive foundation for future research and improving container security in cloud environments.

CCS Concepts: • **Cloud computing** → **Virtualized systems**; *Security*; • **Security and privacy** → Threat modeling;

Additional Key Words and Phrases: Containerization security, cloud computing, confidential computing, vulnerabilities, hardware-based security

ACM Reference Format:
Omar Jarkas, Ryan Ko, Naipeng Dong, and Redowan Mahmud. 2025. A Container Security Survey: Exploits, Attacks, and Defenses. *ACM Comput. Surv.* 57, 7, Article 170 (February 2025), 36 pages. https://doi.org/10.1145/3715001

The authors acknowledge the University of Queensland for strategic funding of the transdisciplinary cyber security research network and thank the UQ Cyber Funding Research Higher Degree Scholarship for supporting Omar Jarkas's work as part of his Doctor of Philosophy (Ph.D.).
Authors' Contact Information: Omar Jarkas (Corresponding author), The University of Queensland, Brisbane, Australia; e-mail: o.jarkas@uq.edu.au; Ryan Ko, The University of Queensland, Saint Lucia, Australia; e-mail: ryan.ko@uq.edu.au; Naipeng Dong, The University of Queensland, Saint Lucia, Australia; e-mail: n.dong@uq.edu.au; Redowan Mahmud, Curtin University, Perth, Australia; e-mail: Mdredowan.Mahmud@curtin.edu.au.

ACM 0360-0300/2025/02-ART170
https://doi.org/10.1145/3715001

1 Introduction

In today's digital transformation era, cloud computing plays a pivotal role. It offers scalable and flexible resources, with virtualization being a critical technology that optimizes the use of these resources. Containerization takes virtualization a step further. It enhances scalability and reduces the resources needed. It achieves this by bundling code, data, and dependencies into containers at the **operating system (OS)** level. These containers share a single kernel, eliminating the need for separate OSs for each application [1]. This advancement offers advantages over virtual machines (VMs), such as improved operational efficiency and deployments [2, 3].

However, the growth of container technology, especially in cloud and shared environments, has revealed significant security risks. These risks stem from the shared kernel model and the overlap of dependencies. These weaken the isolation between containers and increase the potential for attacks [4, 5]. Additionally, the complex and temporary nature of the container challenges conventional security methods. This challenge underscores the critical need for advanced layered security strategies tailored to the unique demands of containerized systems [6, 7].

This study analyzes more than 200 container-related vulnerabilities to advance container security. We introduce a novel classification method, categorizing vulnerabilities by exploit types, attack vectors, and targeted mitigation strategies. This approach provides a comprehensive security framework that spans kernel to application levels, clarifying container-specific challenges and critical vulnerabilities. By mapping these to software and hardware solutions, we evaluate the effectiveness of current measures and propose an integrated strategy to address identified gaps. Our research yields 47 types of exploit, 11 categories of attacks, and 10 specific mitigation strategies (6 based on software and 4 on hardware) to improve container security.

This approach addresses complex risks in the container ecosystem, contributing to container security. By addressing a wide range of security challenges and proposing solutions, our research aims to bolster the resilience of containerized systems, especially in complex, multi-tenant, and cloud environments that demand robust security measures. The contributions are as follows:

(1) **Dive into vulnerabilities:** Deep analysis of 200+ container-related vulnerabilities, classified into 47 exploit types and 11 attack vectors for clear understanding.
(2) **Holistic security mapping:** A comprehensive mapping of exploits and attacks to software and hardware mitigation strategies, offering a complete view of best practices and countermeasures.
(3) **Guiding future research:** Insights derived from our analysis highlight areas that require further investigation, thus setting a direction for future advances in container security.

Our study improves container security by analyzing over 200 vulnerabilities and introducing a framework covering kernel to the application layers. We check security measures by mapping these vulnerabilities to relevant software and hardware solutions and propose an integrated strategy to address gaps. This strengthens container resilience in complex, multi-tenant environments. Our method combines extensive data collection with a systematic review of academic and industry sources, making this study more applicable than earlier research. Unlike previous work, we integrate academic insights with real-world vulnerabilities to provide an actionable framework for improving container security. This study is a pivotal reference for addressing key security challenges and proposing effective strategies. See Table 1 for a comparative analysis with prior surveys. Sultan et al. [4] offered a broad overview but lacked a detailed vulnerability analysis. Pahl et al. [18] focused on select studies without addressing broader security issues. Bélair et al. [9] created a defense taxonomy but missed comprehensive classifications. Gholami et al.

Table 1. Comparative Analysis of Our Work with Other Contemporary Surveys and SoKs

Issues	Challenges Systematic Analysis	Mitigation Strategies Holistic Review	Empirical Data Integration	Hardware and Software Cross-comparative Analysis	Threats and Defenses Taxonomy	Future-oriented Perspectives
Sultan et. al [4]	◐	◐	◐	◐	×	◐
Pahl et. al [8]	◐	◐	◐	×	×	◐
Bélair et al. [9]	✓	◐	◐	×	✓	✓
Gholami et al. [10]	×	×	◐	×	×	×
Yang et al. [11]	◐	◐	◐	◐	◐	◐
Desai et al. [12]	✓	◐	◐	◐	✓	✓
Tosatto et al. [13]	◐	◐	×	◐	×	×
Watada et al. [14]	◐	◐	◐	◐	×	◐
Arriaga et al. [15]	◐	◐	◐	◐	×	✓
Fei et al. [16]	×	×	◐	◐	×	×
Wong et al. [17]	✓	✓	×	×	✓	✓

Key: ✓ - Covered, ◐ - Partially Covered, × - Not Covered.

[10] and Yang et al. [11] discussed security without mapping vulnerabilities to specific mitigations. Lin et al. [19] focused on Docker and kernel attacks, and Wong et al. [17] offered a broad survey but lacked the detailed categorization crucial for formulating nuanced measures. Table 1 highlights how our work stands out from prior surveys through the collection of a dataset that bridges academic research and industry practice, offering a detailed view of container security vulnerabilities.

We compiled a vulnerability dataset that has significant impact on container security; steps include:

- *Keyword Development:* We generated a container-related vulnerabilities keywords list, including *container escape*, *image tampering*, *runtime vulnerabilities*, *privilege escalation*, *namespace bypass*;
- *Literature Search:* Using these keywords, we performed systematic searches in academic databases (ScienceDirect, ACM, IEEE) and collected articles based on date (2013–2024) and relevance. We collected over 100 peer-reviewed articles that meet our criteria for relevance and quality;
- *Industry Sources:* We review leading technology vendors such as Docker, Podman, and Kubernetes to ensure a representative collection of industry insights alongside academic perspectives;
- *Automated CVE Collection:* We developed a script to automate the collection of **Common Vulnerabilities and Exposures (CVE)** entries from the **National Vulnerability Database (NVD)** [20, 21] that matched our predefined keywords;
- *Vulnerability Inclusion Criteria:* We included **Common Vulnerability Security Score (CVSS)** critical vulnerabilities that impacted containers, published between 2013 and 2024, with detailed documentation, including descriptions, impact assessments, and proofs of concept where available;
- *Data Filtering and Cleaning:* We filtered out irrelevant or duplicate entries;
- *Dataset Compilation:* We compiled a dataset of over 200 vulnerabilities, which included detailed information such as CVE ID, description, affected components, severity score, and references;
- *Validation:* We cross-referenced each vulnerability with the literature for consistency.

Table 2 outlines the study's structure, segmenting the container architecture into layers ($\mathcal{L}$). Section 3, the core of our classification, details attack types ($\mathcal{A}$) and exploits ($\mathcal{E}$). For instance,

Table 2. A Comprehensive Overview of the Article: Exploits, Attacks, Mitigations, and Lessons Learned

Sections	Subsection	Attacks/Strategies	Exploits/Mitigations
Introduction Section 1 & Background Section 2	Container Architecture $\mathcal{L}$ Section 2	Orchestration Layer $\mathcal{L}_1$	
		Application Layer $\mathcal{L}_2$	
		Engine & Runtime Layer $\mathcal{L}_3$	
		Host Layer $\mathcal{L}_4$	
		Hardware Layer $\mathcal{L}_5$	
Attacks, Exploits, & Vulnerabilities Section 3	Orchestration Layer $\mathcal{L}_1$ Section 3.1	Orchestration Attacks $\mathcal{A}_1$	Binary $\mathcal{E}_1$, Control Plane $\mathcal{E}_2$, Parsing $\mathcal{E}_3$
			File $\mathcal{E}_4$, Secrets $\mathcal{E}_5$, Overlay Network $\mathcal{E}_6$
			Traffic Redirection $\mathcal{E}_7$
	Application Layer $\mathcal{L}_2$ Section 3.2	Application-level Attacks $\mathcal{A}_2$	Input $\mathcal{E}_8$, File $\mathcal{E}_9$
			Overflows $\mathcal{E}_{10}$, Injection $\mathcal{E}_{11}$
			Default Misconfigurations $\mathcal{E}_{12}$
	Engine Layer $\mathcal{L}_3$ Section 3.3	Image Integrity Attacks $\mathcal{A}_3$	Base Image $\mathcal{E}_{13}$, Image Spoofing $\mathcal{E}_{14}$
			Build $\mathcal{E}_{15}$, Registry $\mathcal{E}_{16}$
		Container Run-time Attacks $\mathcal{A}_4$	System $\mathcal{E}_{20}$, Policy $\mathcal{E}_{17}$
			Privilege $\mathcal{E}_{18}$, Daemon $\mathcal{E}_{19}$
		System Attacks $\mathcal{A}_5$	System $\mathcal{E}_{20}$, Patch $\mathcal{E}_{21}$
	Host Layer $\mathcal{L}_4$ Section 3.4	PID namespace attacks $\mathcal{A}_{6.1}$	FD $\mathcal{E}_{22}$, Process $\mathcal{E}_{23}$
		Network Namespace Attacks $\mathcal{A}_{6.2}$	Memory $\mathcal{E}_{24}$, Netfilter $\mathcal{E}_{25}$
			Network Protocol $\mathcal{E}_{26}$
		Mount Namespace Attacks $\mathcal{A}_{6.3}$	Symlink $\mathcal{E}_{27}$, Memory $\mathcal{E}_{28}$
			Isolation $\mathcal{E}_{29}$, Access $\mathcal{E}_{30}$
		User Namespace Attacks $\mathcal{A}_{6.4}$	UID $\mathcal{E}_{31}$, Privileges $\mathcal{E}_{32}$
		IPC Namespace $\mathcal{A}_{6.5}$	Inter-process Interference $\mathcal{E}_{33}$
		Cgroups Namespace $\mathcal{A}_{6.6}$	Resource Misallocation $\mathcal{E}_{34}$
		System Call Exposure $\mathcal{A}_7$	System Calls Mishandling $\mathcal{E}_{35}$
			Syscall Misimplementations $\mathcal{E}_{36}$
		Virtual File-system $\mathcal{A}_8$	File $\mathcal{E}_{37}$, Path $\mathcal{E}_{38}$
		eBPF $\mathcal{A}_9$	Registers $\mathcal{E}_{39}$, Boundary $\mathcal{E}_{40}$, eBPF FD $\mathcal{E}_{41}$
		Kernel Modules $\mathcal{A}_{10}$	Device Driver $\mathcal{E}_{42}$, Side-Channels $\mathcal{E}_{43}$
	Hardware Layer $\mathcal{L}_5$ Section 3.5	Hardware-based Attacks $\mathcal{A}_{11}$	Speculative $\mathcal{E}_{44}$, Cache $\mathcal{E}_{45}$, MDS $\mathcal{E}_{46}$
			Power & Timing $\mathcal{E}_{47}$
Container Mitigation Strategies Section 4	Software-based $\mathcal{S}$ Section 4.1	Security & Integrity $\mathcal{S}_1$	File System $\mathcal{M}_1$
			Integrity & Allow-Listing $\mathcal{M}_2$
			File System $\mathcal{M}_3$
			File & Symlink Protection $\mathcal{M}_4$
			Image Scanning $\mathcal{M}_5$
			Verification $\mathcal{M}_6$
			Trusted Source Utilization $\mathcal{M}_7$
			Configuration & Handling $\mathcal{M}_8$
		Runtime & Isolation $\mathcal{S}_2$	Container Runtime $\mathcal{M}_9$
			Resource Management $\mathcal{M}_{10}$
			Runtime Monitoring $\mathcal{M}_{11}$
			Container Communications $\mathcal{M}_{12}$
		Kubernetes Enhancements $\mathcal{S}_3$	RBAC $\mathcal{M}_{13}$, API $\mathcal{M}_{14}$
			Authentication $\mathcal{M}_{15}$
			Key & Secrets Managements $\mathcal{M}_{16}$
			Networking Policy $\mathcal{M}_{17}$
		Network Encryption $\mathcal{S}_4$	Encryption Practices $\mathcal{M}_{18}$
			Traffic Management $\mathcal{M}_{19}$
			Network Safeguards & Audits $\mathcal{M}_{20}$
		Coding & Design $\mathcal{S}_5$	Secure Coding Practices $\mathcal{M}_{21}$
			Policies & Configurations $\mathcal{M}_{22}$
			WAFs & Runtime Protection $\mathcal{M}_{23}$
			Data Processing $\mathcal{M}_{24}$
			Management & Auditing $\mathcal{M}_{25}$
			Coding & Patch Management $\mathcal{M}_{26}$
		Hardware Enhancements $\mathcal{S}_6$	Hardware & Firmware $\mathcal{M}_{27}$
			Speculative Execution $\mathcal{M}_{28}$
			Timing & Power Analysis $\mathcal{M}_{29}$
	Hardware-based $\mathcal{H}$ Section 4.2	Trusted Platforms $\mathcal{H}_{30}$, TEEs $\mathcal{H}_{31}$	
		Trusted Domains $\mathcal{H}_{32}$	
		Hardware Virtualization Support $\mathcal{H}_{33}$	
Lessons Learned Section 5		Linux Kernel	Security in Contextualization
			Container-Specific Security
			Role of IMA in Containers
			Adapting Frameworks
			Challenges with IMA
			Comprehensive Security
		eBPF	Importance & Risks
		Confidential Computing	Integration in Containers

in the Orchestration Layer ($\mathcal{L}_1$), attacks include binary exploits ($\mathcal{E}_1$), control plane compromises ($\mathcal{E}_2$), and parsing vulnerabilities ($\mathcal{E}_3$). This section classifies 11 attacks by mechanism, linking them to exploits and vulnerabilities. Subsequent sections cover mitigations, with hardware ($\mathcal{S}$) and software ($\mathcal{M}$) approaches in Section 4. Section 5 discusses findings and future research, followed by conclusions in Section 7.

2 Background

This section introduces the fundamentals of containerization, highlighting its key roles and challenges in modern cloud computing and underscoring the need for comprehensive security strategies.

Hypervisor-based virtualization involves creating VMs at the hardware level [15, 22]. Each VM acts as a complete physical computer, allowing multiple OSs to run on a single server. A hypervisor (e.g., VMware ESXi, Microsoft Hyper-V, Oracle VirtualBox) manages the interaction between VMs and the underlying hardware [23–25]. This method offers strong isolation but introduces overhead due to running an OS within each VM, limiting efficiency and scalability [2].

Containerization, in contrast, is more efficient [3]. Containers encapsulate an application and its necessary components, sharing the host OS kernel for isolation and effectiveness. This eliminates the need for an individual OS in every virtual instance, leading to faster deployment, increased flexibility, and improved resource utilization. Containerization has revolutionized software development and deployment by offering lightweight, portable environments that ensure consistent application behavior across diverse infrastructures, promoting rapid scalability and simplified movement between cloud and on-premise systems. Technologies such as Docker [1] and Kubernetes [26] have become essential for microservices and cloud-native architectures, providing powerful tools for container creation, orchestration, and scaling.

Despite their advantages, containers present security challenges. Their shared kernel architecture weakens isolation, increasing the attack surface and making them vulnerable to privilege escalation exploits [4, 5, 19]. The complex interactions between containers, orchestrators, and underlying infrastructure necessitate a multilayered security approach. Prioritizing security is paramount in containerized environments, requiring both software and hardware-based solutions to mitigate risks from the application layer down to the kernel and container engine.

The containerized environment comprises several layers, as shown in Figure 1. Understanding these layers is crucial for ensuring security, as each has unique functions and vulnerabilities, creating a complex landscape of potential attacks, exploits, and mitigation strategies. This figure visually encapsulates the interconnected nature of container vulnerabilities and their mitigation, illustrating the multi-layered approach necessary for comprehensive security.

Orchestration Layer $\mathcal{L}_1$ manages container operations across platforms such as Kubernetes, Docker Swarm, and Apache Mesos [27–31], automating deployment, scaling, and management of containerized applications to promote efficient resource use and high availability. However, its centrality exposes it to security threats. Misconfigurations, software vulnerabilities, and weak access controls can lead to unauthorized access and data breaches (see Section 3.1). Securing $\mathcal{L}_1$ requires protecting the cluster and control plane, enforcing least privilege, hardening network configurations, implementing strong authentication and authorization, and continuous monitoring and audits.

Application Layer $\mathcal{L}_2$ is where applications are executed along with their code, dependencies, and runtime configurations. While containers offer isolation, $\mathcal{L}_2$ remains vulnerable to attacks targeting code flaws and outdated dependencies [4]. Insecure coding practices, unpatched components, poor input validation, and misconfigurations can lead to unauthorized access, data breaches, or **arbitrary code execution (ACE)** (see Section 3.2). Mitigating these risks requires secure development practices, dependency management, rigorous input validation, and secure-by-default configurations (see Section 4.1.5). Collaboration between developers and security teams is critical for ongoing security.

Engine and Runtime Layer $\mathcal{L}_3$ is the core of container operations. Engines (e.g., Docker) and runtimes (e.g., runc, containerd) manage container lifecycles from image creation to execution using

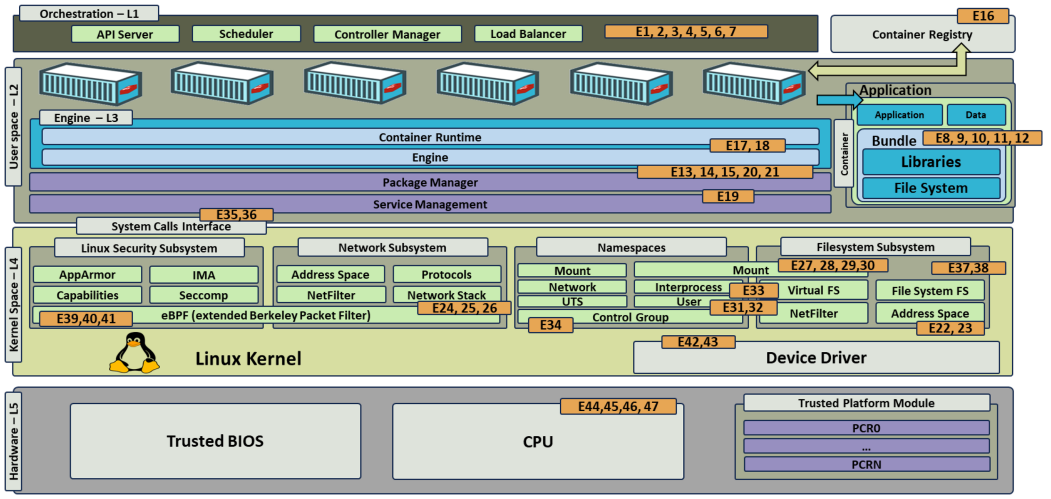

Fig. 1. Mapping of the exploits to the different container architectural layers.

namespaces and cgroups for isolation [32–36]. However, interaction with the host kernel creates potential attack vectors, such as privilege escalation or container escapes (see Sections 3.3.1–3.3.3). Securing $\mathcal{L}_3$ requires ensuring container image integrity, applying least privilege principles, restricting actions through engine and runtime configurations, and securing the API [1]. Regular updates to engines, runtimes, and their dependencies are crucial (see Sections 4.1.1 and 4.1.2). Hardening the host kernel is also essential to prevent attacks that exploit this layer [29, 37].

Host Layer $\mathcal{L}_4$ plays a critical role in the security of containerized environments due to containers sharing the host kernel, introducing unique vulnerabilities [38] (see Sections 3.3.1–3.3.3). Namespace isolation is vital for separating container environments and partitioning kernel resources, but it is not foolproof against all attack vectors. Each namespace type (PID, network, mount, user, IPC, UTS, and cgroups) faces specific threats that require custom mitigation strategies. The transient nature of containers, with rapid state and configuration changes, adds complexity to $\mathcal{L}_4$ security. Addressing these challenges requires comprehensive and adaptive security measures, including stringent access controls, container-specific security tools, and best practices in container configuration and management (see Section 4.1.2). Continuous monitoring and real-time analysis are essential for identifying and responding to security threats. Also, hardware-based security can enhance the security of the underlying host by using techniques such as **Trusted Execution Environments (TEEs)**, trusted domains, and implementing a trusted platform to ensure integrity (see Section 4.2).

Hardware Layer $\mathcal{L}_5$ underpins computing, integrating essential components such as CPUs, memory, and other hardware crucial for operations. Securing this layer in cloud computing and containerized environments is imperative, because vulnerabilities could compromise the integrity and isolation of applications [19]. Advances in hardware, such as speculative execution and sophisticated cache hierarchies, enhance performance and efficiency but introduce new security vulnerabilities. These include attacks that exploit processor optimizations for information leakage, security bypass, and unauthorized execution, particularly problematic in multi-tenant cloud setups [39, 40]. Addressing these threats demands a collaborative effort: Manufacturers need to refine designs and issue firmware updates; OSs must adopt safeguards such as kernel page table isolation; developers need to adhere to secure coding practices; and administrators should use configurations that reduce side-channel risks (e.g., hyperthreading adjustments) [19, 39, 40].

These layers highlight the complexity of securing containers; security solutions must protect all layers to avoid inefficiencies and new vulnerabilities. As container technologies evolve, a deeper understanding of specific threats and their exploits is paramount. This study addresses this crucial need with data-driven analysis, providing actionable recommendations for enhanced security.

3 Attacks, Exploits, and Vulnerabilities

This section analyzes over 200 container-related vulnerabilities, organizing them into 11 attack types ($\mathcal{A}$) and their corresponding exploits ($\mathcal{E}$) based on root causes. This taxonomy clarifies the complex threats in container ecosystems and aids in developing targeted security measures ($\mathcal{M}$) (see Section 4). The analysis details these 11 attack categories, covering exploits and vulnerabilities across layers from orchestration to host/kernel and hardware.

Our taxonomy was developed through an iterative process consisting of the following steps: We began by grouping the collected vulnerabilities based on the layer of the container stack they affect. For each vulnerability within a layer, we analyzed its characteristics, including its impact, attack vector, affected components, and root cause. We grouped vulnerabilities that share common root causes and attack vectors into exploit categories $\mathcal{E}$. For example, vulnerabilities that enable container escape via namespace manipulation were grouped together. We further aggregated the exploit categories into broader attack types $\mathcal{A}$ based on the underlying mechanisms and objectives of the exploits. This step involved identifying patterns and commonalities among the exploits. We reviewed and refined the groupings by cross-referencing with existing literature, security reports, and expert feedback. This iterative process involved re-evaluating the classifications to ensure accuracy and comprehensiveness. We documented the hierarchical structure of layers, attack types, and exploits, providing clear definitions and examples for each category. Figure 2 presents a hierarchical structure of layers, attacks, and exploits, offering deeper insight into the relationships between vulnerabilities and their impact on container security. This classification taxonomy aids in visually understanding how different security measures interact across various layers, enhancing the ability to navigate the complex security dynamics of containerized environments.

3.1 Orchestration Layer $\mathcal{L}_1$

We examine the security challenges at $\mathcal{L}1$, identifying vulnerabilities leading to $\mathcal{A}1$. This sets the stage for a detailed analysis of exploits $\mathcal{E}1$ to $\mathcal{E}7$, highlighting the need for robust defense strategies.

Orchestration Attacks $\mathcal{A}_1$. Improper Handling of Malicious Binaries $\mathcal{E}_1$ presents security risks when orchestration-level commands are used for file transfers between containers and the host. Attackers can replace legitimate binaries (e.g., `tar`) with malicious versions [27], facilitating ACE and system tampering. CVE-2019-11246 and CVE-2019-11249 highlight these risks due to inadequate validation [41], emphasizing that even with user permissions, severe consequences such as unauthorized access or system takeover can occur.

Control Plane Bypassing $\mathcal{E}_2$ targets vulnerabilities in control plane components such as the API server and admission controllers [27]. Attacks can lead to unauthorized access, privilege escalation, or **denial-of-service (DoS)**, allowing attackers to seize system control. Examples include CVE-2020-8552, CVE-2021-25735, CVE-2018-1002105, CVE-2023-3978, and CVE-2020-8559 [42–44]. Causes include inadequate rate limiting, unvalidated redirects, mishandling of **Transport Layer Security (TLS)** [45] credentials, and insufficient webhook validation.

Input and Parsing Mishandling $\mathcal{E}_3$ involves security issues from incorrect parsing of YAML and JSON, leading to DoS attacks by overloading CPU or memory [46, 47]. CVE-2019-11253 highlights poor payload management, and CVE-2019-11254 points to YAML parsing inefficiencies. CVE-2021-25738 demonstrates risks in tools like the Kubernetes Java Client library, where loading YAML

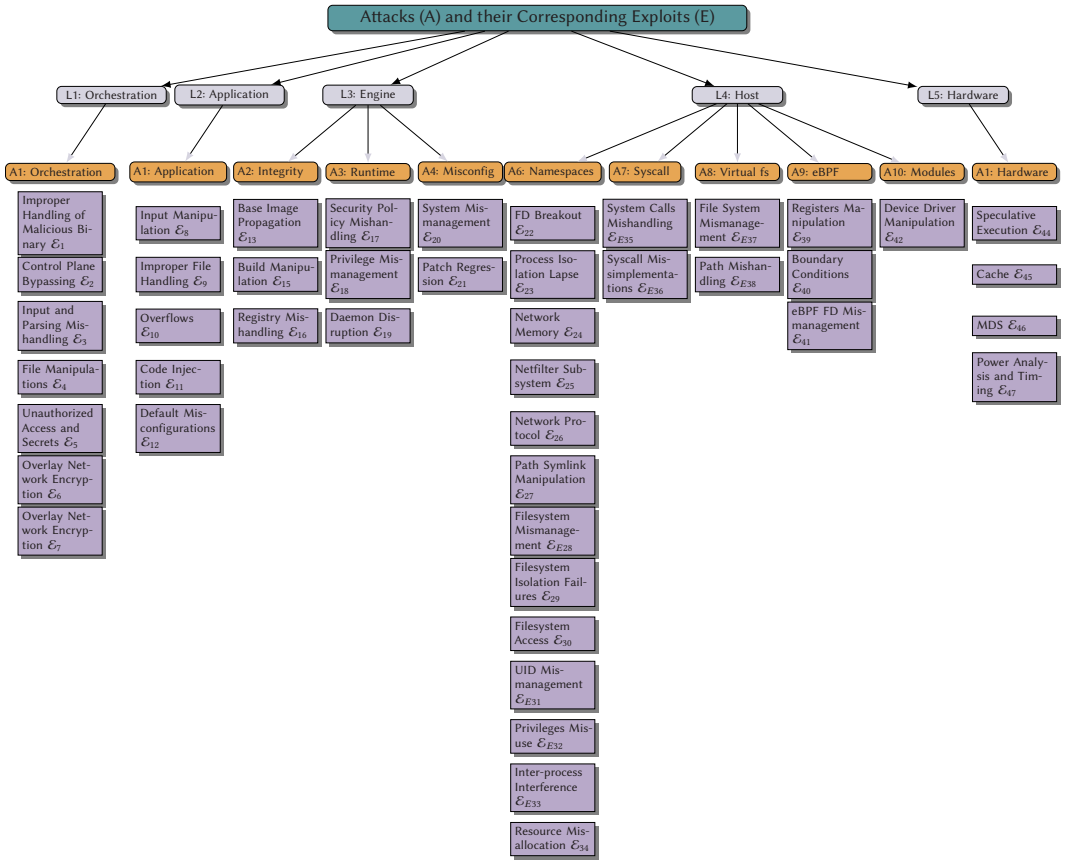

Fig. 2. Mapping of Layers $\mathcal{L}$, attacks $\mathcal{A}$, exploits $\mathcal{E}$.

files can lead to ACE due to unsafe deserialization [46, 48]. These examples emphasize the need for validation, secure coding, and efficient data processing.

File Manipulations $\mathcal{E}_4$ stem from improper handling of volumes, file permissions, symlinks, and temporary storage in containerized environments. Notable examples include CVE-2020-8557, where Kubernetes fails to limit disk usage for pod-generated files like `/etc/hosts`, causing node failure. CVE-2019-1002101 and CVE-2017-1002102 highlight insecure volume management, leading to risks of arbitrary file deletion or unauthorized host filesystem access [49]. These issues underscore the importance of secure configurations and vigilant coding practices to prevent node failures, ACE, and loss of critical files.

Unauthorized Access and Secrets $\mathcal{E}_5$ expose sensitive data and functionalities due to specific vulnerabilities. Examples include CVE-2020-8565, where improper logging reveals tokens in log files; CVE-2021-25742, allowing access to cluster secrets through misuse of ingress-nginx custom snippets; and CVE-2020-8558, where a Kubelet `kube-proxy` flaw makes services on `127.0.0.1` accessible to nearby hosts. These examples underscore the importance of secure token handling, secret management, and prevention of service exposure through rigorous design and configuration.

Overlay Network Encryption $\mathcal{E}_6$ addresses vulnerabilities in overlay network encryption, essential for secure container-to-container communication within **virtual LANs (VLANs)**. The Moby project, foundational to Docker Engine, identifies CVE-2023-28840, CVE-2023-28841, and

CVE-2023-28842 affecting the overlay network driver critical to Swarm Mode. These vulnerabilities include incorrect rule precedence allowing unencrypted datagrams to bypass security, enabling unencrypted data transmission within secure networks, and improper kernel configuration permitting cleartext datagrams, risking data breaches and DoS. These issues highlight the importance of careful encryption mechanism design, configuration, and validation in overlay networks [50, 51].

Traffic Manipulation and Redirection $\mathcal{E}_7$ stem from vulnerabilities permitting unauthorized traffic control, redirection, and manipulation. Vulnerabilities such as CVE-2020-10749 exploit IPv6 router advertisements for **man-in-the-middle (MITM)** attacks; CVE-2021-25737 enables redirection of pod traffic to node-private networks; CVE-2022-3172 and CVE-2020-8554 involve unauthorized URL redirections and traffic interceptions due to misconfigurations in ClusterIP services. CVE-2019-9946 addresses firewall rule misconfigurations affecting traffic routing. CVE-2021-25740 and CVE-2022-3294 showcase confused deputy attacks, where attackers trick a legitimate authority into misdirecting traffic or accessing unauthorized resources. These vulnerabilities undermine network communication trust, allowing attackers to manipulate control plane network statuses or breach secure endpoints. Effective mitigation involves strong network policies, traffic encryption, service mesh architectures, and stringent endpoint security.

3.2 Application Layer $\mathcal{L}_2$

We explore security challenges at $\mathcal{L}2$, identifying vulnerabilities leading to $\mathcal{A}2$, laying the foundation for a detailed investigation into their nature and origins. Emphasizing the need for robust defense strategies, we highlight the importance of effective risk mitigation against these threats.

Application-level Attacks $\mathcal{A}_2$. Building on the overview, we now focus on $\mathcal{A}_2$, examining the range of exploits from $\mathcal{E}_8$ to $\mathcal{E}_{12}$. This detailed analysis delves into the mechanisms and specificities of exploiting these vulnerabilities, aiming to provide insights into targeted defense measures.

Input Manipulation $\mathcal{E}_8$ related to containerized applications involving attackers manipulating or injecting malicious input to start remote ACE. Various root causes, such as insecure input validation, improper data deserialization, mismanagement of scripting engines, unsafe service configurations, and insufficient sandboxing of external input, can cause these vulnerabilities [52]. For instance, CVE-2015-8103 in Jenkins allows attackers to exploit data deserialization by executing arbitrary commands, while CVE-2015-1427 and CVE-2014-3120 use the platform's scripting capabilities to execute arbitrary code. CVE-2017-12149 allows remote authenticated users to execute arbitrary code using crafted serialized data. Parsing or processing errors can enable attackers to inject and execute malicious commands, as seen with CVE-2017-5638 in Apache Struts and CVE-2014-6271, also known as ShellShock in GNU Bash [53]. Furthermore, insecure configurations and lack of input sanitation can lead to executing arbitrary commands when attackers supply files or input, such as CVE-2017-8291 and CVE-2016-3714. CVE-2017-12615 and CVE-2016-3088 allow attackers to upload and run files. CVE-2019-11043 highlights flaws in processing HTTP requests, resulting in ACE. The interconnected operations of containers can amplify the consequences of these vulnerabilities, leading to unauthorized access, data compromise, privilege escalation, or even host takeover.

Improper File Handling $\mathcal{E}_9$ in containerized environments stems from flawed file handling and inadequate access controls, exposing systems to manipulation by malicious actors. Symlink attacks, such as those identified in CVE-2016-1240, CVE-2016-1247, and CVE-2016-9566, enable privilege escalation by misdirecting file operations. Furthermore, vulnerabilities such as CVE-2018-1313 (password hash retrieval), CVE-2017-7529 (integer overflow leading to information disclosure), and CVE-2015-5531 (data exposure via directory traversal) highlight the risks of unintentional

information disclosure. Given the shared OS kernel and elevated privileges common in containers, such exploits can threaten individual containers and the entire host system or cluster.

Overflows $\mathcal{E}_{10}$ occur when programs manage memory operations with external data, leading to buffer overflows and overreads. Such vulnerabilities can cause unexpected behavior, unauthorized data access, or total system compromise. Containerized environments magnify the impact, as a single compromised container can affect others, heightening the risk of widespread breaches. CVE-2016-5636 and CVE-2017-1000158 highlight the threat of integer overflows leading to heap-based buffer overflows, enabling attackers to execute arbitrary code, escalate privileges, or access sensitive data. The *Heartbleed* bug (CVE-2014-0160) in OpenSSL demonstrates the severe implications of buffer overreads, allowing attackers to extract private data by exploiting SSL/TLS protocol vulnerabilities. Additionally, CVE-2014-0050 showcases the dangers of failing to handle data boundaries, where malicious input can cause service-denying infinite loops. The interconnectedness of containerized systems amplifies these risks, allowing a single breach to compromise the broader infrastructure.

Code Injection $\mathcal{E}_{11}$ in web applications on containerized systems allows adversaries to inject malicious code or commands, leading to unauthorized access, data breaches, or other adverse activities. CVE-2020-15227 exemplifies this, where attackers exploit input validation flaws for code injection. Similarly, CVE-2017-8917 demonstrates how an SQL injection can enable unauthorized database access through SQL commands. CVE-2016-9920 underscores the danger of inadequate input handling, allowing remote execution of commands. The interconnectedness of such environments amplifies these risks, with a breach in a single container posing a threat to the entire system.

Default Misconfigurations $\mathcal{E}_{12}$ can happen when administrators do not configure settings, leaving applications, data, or the system open to attack. CVE-2018-1313 shows the danger of revealing confidential data due to incorrect database permissions, such as password hashes. The Python `urllib` vulnerability (CVE-2019-9948) arises because default support for `local_file:scheme` can lead to local file inclusion attacks. In the case of Django (CVE-2017-12794), a misconfiguration of the technical display could make sensitive debug information available to attackers. The consequences of vulnerabilities can be severe in a containerized environment: Containers separate applications and their related run-time environments by design. Yet, misconfigurations can break down this isolation, allowing unauthorized data access, privilege escalation, or even movement between containers. The default settings rank ease of deployment over security and can worsen these risks.

3.3 Engine Layer $\mathcal{L}_3$

We explore $\mathcal{L}3$, identifying vulnerabilities leading to $\mathcal{A}3$, $\mathcal{A}4$, and $\mathcal{A}5$. Emphasizing defense strategies, we highlight the need for mitigation to protect against these threats.

3.3.1 Image Integrity Attacks $\mathcal{A}_3$. $\mathcal{A}3$ encompasses vulnerabilities exploiting weaknesses in image handling, verification, and management, identified as exploits $\mathcal{E}13$ to $\mathcal{E}_{16}$. These can lead to ACE, spoofing, and other activities compromising container security.

Base Image Propagation $\mathcal{E}_{13}$ highlights risks inherent in widely adopted base container images, which can introduce system-wide vulnerabilities. CVE-2018-11757 showcases risks from outdated Docker tags allowing ACE, while CVE-2020-35467 and CVE-2020-35197 point out dangers of default root passwords. These examples emphasize the importance of careful image selection and configuration to prevent broad system compromises.

Image Spoofing $\mathcal{E}_{14}$ exploits identification and validation weaknesses in container management, employing tactics such as repository spoofing, path traversal, image cache poisoning, and DoS

attacks. Vulnerabilities such as CVE-2015-9258 and CVE-2015-9259 expose the lack of trust mechanisms for root file verification. CVE-2014-5282 and CVE-2014-9358 reveal challenges with image ID validation, posing spoofing risks. CVE-2014-8179 and CVE-2017-14992 highlight improper manifest handling and gzip bombing threats from unverified content, stressing the importance of robust verification processes.

Build Manipulation $\mathcal{E}_{15}$ involves vulnerabilities during the building, extracting, and manipulating phases of images, resulting in malicious code injection, privilege escalation, or unauthorized command execution. Addressing these risks requires confirming the integrity of source code, binaries, and archives. CVE-2019-13139, where improper Git URL handling led to ACE, and CVE-2014-9357, where crafted archives enabled privilege escalation, highlight the need for security controls and verification throughout the container build process.

Registry Mishandling $\mathcal{E}_{16}$ involves vulnerabilities in container image registries leading to unauthorized privilege escalations, DoS attacks, ACE, and exploitation of insecure communication protocols. Examples include CVE-2019-16097, where inadequate permission management allowed non-admin users to gain admin rights; CVE-2017-11468, exposing systems to DoS due to uncontrolled data intake; CVE-2021-29641, highlighting ACE through flawed file-upload permissions; and CVE-2015-1843, revealing MITM attacks via insecure HTTPS to HTTP transitions. These cases underscore the importance of proper permission management and secure registry configurations.

3.3.2 Container Run-time Environment $\mathcal{A}_4$. $\mathcal{A}4$ encompasses security breaches within container environments due to mismanagement of configurations and security policies ($\mathcal{E}17$), flawed access control or privilege handling ($\mathcal{E}18$), and daemon disruptions ($\mathcal{E}19$).

Security Policy Mishandling $\mathcal{E}_{17}$ compromises container integrity through improper configurations, lax security policy management, and incorrect handling of security mechanisms such as **Linux Security Modules (LSM)**, AppArmor profiles, and mount targets. For instance, CVE-2015-3631 reveals how Docker Engine's LSM and `docker_t` policies could be exploited for privilege escalation. CVE-2016-8867 shows Docker Engine's susceptibility to user permission bypasses by malicious images, compromising filesystem integrity. CVE-2019-16884 in runc and CVE-2017-6507 involving Docker and LXD illustrate AppArmor profile bypasses and mishandling, increasing the attack surface. These vulnerabilities highlight the need for configuration and security policy enforcement.

Privilege Mismanagement $\mathcal{E}_{18}$ in technologies such as Docker Engine [50], runc [34], containerd [35], and Kata Containers [54] can lead to unauthorized access and privilege escalation through misconfigurations and inherent flaws. Vulnerabilities such as CVE-2022-0811, CVE-2020-27151, CVE-2020-13401, CVE-2019-19921, CVE-2020-15257, and CVE-2022-23648 highlight issues with inadequate access controls, exposed Unix sockets, and improper image configuration and kernel option management. These issues broaden attack surfaces, enabling activities such as container escape, ACE, and information disclosure, compromising integrity. In container orchestration systems, these threaten containers and clusters, emphasizing the need for comprehensive security measures.

Daemon Disruption $\mathcal{E}_{19}$ involves exploits against container management daemons, potentially compromising secrets, causing DoS, or leading to system instability due to errors, mishandling, or malicious input. CVE-2019-13509 shows how Docker Engine's debug mode might expose secrets, while CVE-2021-21285 indicates that a malformed Docker image can disrupt the Docker daemon, triggering a DoS. These underscore the need for careful daemon management and input validation.

3.3.3 System Initialization and Management Misconfigurations $\mathcal{A}_5$. $\mathcal{A}5$ refers to security vulnerabilities stemming from improper setup and oversight of system components and management tools, exemplified by $\mathcal{E}20$ and $\mathcal{E}_{21}$, leading to risks such as unauthorized access and failures.

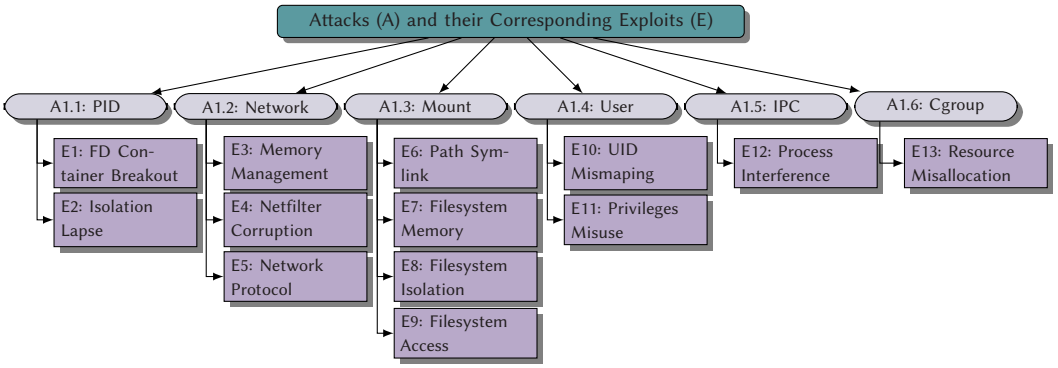

Fig. 3. Mapping of namespace attacks (A) with namespace exploits (E).

System Mismanagement $\mathcal{E}_{20}$ covers vulnerabilities such as CVE-2021-33910 in systemd, allowing OS crashes through memory allocation errors, and CVE-2020-27352, where snapd misconfigurations lead to unauthorized device access due to cgroup delegations. These highlight the complexities of system and container management, emphasizing the need for careful configuration, timely patching, and adherence to best practices to mitigate unauthorized access and system instability.

Patch Regression $\mathcal{E}_{21}$ occurs when updates to components such as Docker and runc reintroduce previously resolved vulnerabilities or miss necessary security fixes, compromising container security. For example, CVE-2020-14300 resurfaced a previously fixed flaw, while CVE-2020-14298 stemmed from an outdated runc version missing the CVE-2019-5736 patch. These cases highlight the risks of poor patch management, underscoring the need for thorough patch verification.

3.4 Host Layer $\mathcal{L}_4$

Host security is critical as the foundation for isolated environments essential for container-based virtualization. This section delves into potential attacks on various host layers, focusing on how vulnerabilities in different host components, such as namespace types $\mathcal{A}_6$, system calls $\mathcal{A}_7$, eBPF, **Virtual File System (VFS)** $\mathcal{A}_8$, and kernel modules $\mathcal{A}_{10}$ pose unique security challenges.

3.4.1 Namespace $\mathcal{A}_6$*.* Namespace isolation is a critical security feature in containerized environments, providing isolated environments within a kernel. This section examines vulnerabilities in this model, focusing on how weaknesses across different namespace types can be exploited, posing unique threats to container security. We will analyze security issues associated with PID, network, mount, user, IPC, UTS, and cgroup namespaces. Each type presents unique vulnerabilities, and our analysis will contribute to understanding defense mechanisms against namespace attacks. Refer to Figure 3 for a detailed mapping of these namespace attacks and corresponding exploits. It graphically demonstrates the exploitation paths and potential security breaches within namespace configurations, further emphasizing the need for robust namespace isolation techniques.

Process ID (PID) *namespace* $\mathcal{A}_{6.1}$*.* The PID namespace maintains process isolation between containers and the host system. Despite its importance, vulnerabilities within the PID namespace can be exploited, resulting in resource access conflicts and potential container escapes. We identify critical exploit categories, including FD Mishandling $\mathcal{E}_{22}$—where flawed handling of **file descriptors (FDs)** may cause resource interference—and Process Isolation Lapse $\mathcal{E}_{23}$, facilitating container escapes.

FD Breakout $\mathcal{E}_{22}$ highlights that in Unix-like systems, such as Linux, FDs are crucial for container interactions with system resources, representing system I/O operations such as files, sockets, and

pipes. Mismanagement of FDs can result in security breaches, including container escapes or unauthorized access to host resources. Examples such as CVE-2016-9962 and CVE-2019-5736 illustrate the risks of inadequate FD handling, from misconfigurations allowing access to the host's filesystem to symbolic link exploitations. Proper FD management and operational sequencing during container setup are essential to avert these issues and maintain container isolation.

Process Isolation Lapse $\mathcal{E}_{23}$ underscores that while process namespaces are crucial for container isolation, actual comprehensive isolation demands integrating mount and user namespaces with cgroups for effective resource management. Implementing stringent security policies to manage resource interactions and access, alongside leveraging LSMs such as SELinux [55] and AppArmor [56], is vital for enhancing isolation [4, 9, 19, 55–59]. Despite these efforts, the complexity of achieving complete isolation can result in vulnerabilities (e.g., CVE-2019-10147, CVE-2019-10145, CVE-2019-10144), highlighting the ongoing challenge of preventing unauthorized access.

Network Namespace $\mathcal{A}_{6.2}$. Network isolation, achieved through network namespaces, is essential for container security. Each container's separate network environment enhances isolation, safeguarding application privacy. However, vulnerabilities in kernel memory management, the Netfilter subsystem, and network protocol implementations can undermine this isolation. These flaws could allow unauthorized access or disrupt communication, compromising container security.

Network Memory $\mathcal{E}_{24}$ underscores the importance of effective network memory management in kernel operations essential for containerized communication, such as memory allocation and deallocation [60, 61]. The complexity of managing network memory makes it prone to vulnerabilities, including double-free issues (e.g., CVE-2017-8890) affecting IPv4 connections and use-after-free problems (e.g., CVE-2017-16939 and CVE-2019-11815) related to XFRM and Reliable Datagram sockets [62]. Such vulnerabilities expose container environments to DoS attacks and unauthorized privilege escalations, threatening container isolation and the security framework. So, meticulous management of network memory is critical to protect container orchestration and maintain the integrity and security of the host system and its interconnected containers.

Netfilter Subsystem $\mathcal{E}_{25}$ plays a role in container security by handling packet filtering, NAT, and port translation, essential for securing communications and enforcing isolation through IP forwarding and routing management [63–65]. Vulnerabilities in Netfilter, such as buffer overflows, use-after-free errors, and other forms of memory corruption, pose risks. CVE-2022-1015, CVE-2022-1016, CVE-2022-32250, CVE-2022-34918, CVE-2021-22555, and CVE-2016-3134, highlight memory corruption issues within packet filtering components. These vulnerabilities, often exploitable by local users who can create user/net namespaces, highlight the critical need for stringent security to enhance Netfilter's defenses, thus strengthening container isolation and security.

Network Protocol $\mathcal{E}_{26}$ emphasizes the crucial role of kernel networking protocols, such as TCP/IP and UDP, in supporting Internet and local network operations vital for container communication and network-intensive applications [66]. Flaws in protocol implementations can expose containerized environments to network attacks, including session hijacking, unauthorized data injection, or DoS flooding. CVE-2016-5696, for example, illustrates the danger of improper TCP ACK segment handling that could allow TCP session hijacking. CVE-2011-0709 demonstrates the potential for DoS attacks from IGMP implementation defects. These vulnerabilities highlight the necessity of meticulous protocol management to protect container security and ensure service continuity.

Mount Namespace $\mathcal{A}_{6.3}$. Mount namespaces are essential for container security. They provide each container with an isolated filesystem, enhancing privacy and preventing unintended file interactions. This isolation safeguards against unauthorized tampering and ensures container activities remain compartmentalized. Yet, Mount Namespace Attacks $\mathcal{A}_{6.3}$ exploit vulnerabilities in this isolation, allowing attackers to escape container boundaries, change files, or gain access.

Path Symlink Manipulation $\mathcal{E}_{27}$ poses a container security risk, where symbolic links and file paths ease file organization and navigation. Improper symlink management or path sanitization can lead to exploits that compromise container isolation and security, allowing attackers unauthorized filesystem traversal, filesystem breakouts, and host compromises. Notable instances include CVE-2021-30465, which involves exploiting insufficient path sanitization; CVE-2014-9356, where a Docker vulnerability allows remote attackers to write to arbitrary files; CVE-2015-3629, a symlink traversal exploit in container libraries enabling namespace escapes; and CVE-2018-15664, exploiting Docker's *cp* command for unauthorized host filesystem access underscoring the necessity for stringent symlink management and path sanitization to protect container integrity and isolation.

Filesystem Memory Mismanagement $\mathcal{E}_{28}$ involves vulnerabilities from improper memory handling in filesystem operations, which are pivotal in containerized settings. These flaws, exemplified by CVE-2021-33909 in BTRFS (out-of-bounds write and root escalation), CVE-2008-5025 in HFS (system crashes from invalid catalog name lengths), and CVE-2019-11487 in FUSE (use-after-free under heavy loads), can lead to unauthorized access, data modification, privilege escalation, or DoS. The prevalence of filesystems such as BTRFS, HFS, and FUSE in containers amplifies the impact of these vulnerabilities, highlighting the need for rigorous memory management to safeguard containers.

Filesystem Isolation Failures $\mathcal{E}_{29}$ create significant security vulnerabilities by enabling unauthorized access, code injection, or bypassing container protections. Such vulnerabilities often arise from the mismanagement of filesystems or libraries, leading to improper interactions with the host or other containers. Notable examples include CVE-2020-2026 and CVE-2020-2023, where compromised containers could execute code on the host or impersonate the Kata agent; CVE-2019-14271, which involves code injection via dynamic library loading in a chroot environment; and CVE-2015-2925, demonstrating a double-chroot attack that circumvents container protections. Issues with filesystem operations, like errors in rename functions within bind mounts, illustrate how local users might overcome security measures. These cases highlight the importance of diligent filesystem isolation practices to safeguard against exploitation and maintain container security.

Filesystem Access $\mathcal{E}_{30}$ is critical for maintaining isolation in containerized environments such as Docker and Kubernetes, leveraging mount namespaces and other mechanisms. Yet, vulnerabilities can emerge from complex permissions, access controls, and resource constraint configurations. Notable vulnerabilities, including path traversal, symlink manipulation, heap overflow, and inadequate permission checks, have been identified in CVEs such as CVE-2015-8660, CVE-2022-0185, CVE-2013-1956, CVE-2022-23648, CVE-2019-16884, CVE-2021-25741, CVE-2017-1002101, CVE-2021-41091, and CVE-2021-3493. These issues often arise from inadequate access controls and flawed validation, allowing unauthorized access or privilege escalation. Effective filesystem management is essential to safeguard containerized applications from these security threats.

User Namespace $\mathcal{A}_{6.4}$. User namespaces strengthen container security by mapping user IDs within containers differently from the host system. This allows processes to have root-level privileges inside the container without compromising host security. However, misconfigurations in user namespaces, such as incorrect UID mapping $\mathcal{E}_{31}$ or overly broad privileges $\mathcal{E}_{32}$, can allow attackers to escalate privileges and potentially gain unauthorized access to sensitive data or systems.

UID Mismanagement $\mathcal{E}_{31}$ involves improper handling, interpretation, or validation of UIDs within containerized systems, leading to privilege escalation, unauthorized access, or arbitrary command execution. Examples include CVE-2021-21284, where Docker's misconfiguration of the `--userns-remap` option grants access to a remapped root; CVE-2016-3697, showcasing runc's mix-up between numeric UIDs and usernames, allowing privilege escalation; and CVE-2020-15257,

pointing out container's insufficient access control for the shim API socket, enabling privilege elevation. These instances underline the importance of accurate UID management for securing containers.

Privileges Misuse $\mathcal{E}_{32}$ occurs when access rights and control permissions and capabilities essential for container isolation are assigned, leading to unauthorized behavior alterations or privilege escalation. Examples include CVE-2019-11328, with insecure permissions enabling unauthorized file changes; CVE-2017-7308, where block size data validation failures occur; and CVE-2016-8655 and CVE-2020-13401, highlighting the dangers of mishandling network capabilities such as `CAP_NET_ADMIN` and `CAP_NET_RAW`, which can enable attacks such as router advertisement spoofing or network manipulation. Further, CVE-2013-1858, CVE-2022-0185, and CVE-2018-14634 expose the consequences of kernel-level permissions mismanagement, such as clone flag mishaps, context issues in filesystems, and ELF table creation flaws, each facilitating privilege escalation.

Interprocess Communication (IPC) *Namespace* $\mathcal{A}_{6.5}$. IPC namespaces are critical for isolating communication mechanisms such as message queues, semaphores, and shared memory segments. This isolation ensures that processes within a namespace can operate, preventing access and maintaining boundaries, which is essential when containers share a kernel. Yet, misconfigurations or kernel vulnerabilities can compromise isolation. Permissive settings or kernel defects might allow unintended access to IPC resources like shared memory, leading to data breaches or unauthorization [67].

Inter-process Interference $\mathcal{E}_{33}$ vulnerabilities arise from improper influence or interference between processes, challenging the isolation provided by IPC namespaces, enabling a malicious entity in one namespace to impact another's process by exploiting shared resources or bypassing security mechanisms. CVE-2022-0847, involving uninitialized flags in pipe buffer structures, and CVE-2021-43267, related to manipulating IPC messages, exemplify such vulnerabilities. These examples underscore the importance of managing shared resources and enforcing access controls to preserve process isolation and security in systems utilizing IPC namespaces.

Control Group (cgroups) *Namespace* $\mathcal{A}_{6.6}$. Cgroup namespaces are pivotal for managing system resources (CPU, memory, network bandwidth, etc.) and enabling administrators to set resource limits. This control and isolation of resource usage across containers are crucial, especially in multitenant environments, to ensure performance stability and security through defined resource allocation boundaries for each container. Yet, vulnerabilities stemming from incorrect cgroup configurations, mismanagement, or design flaws can lead to performance issue and privilege escalation.

Resource Misallocation $\mathcal{E}_{34}$ results from improper allocation, handling, or restriction of system resources by cgroups, designed to limit and isolate resource usage (CPU, memory, disk I/O, etc.) for processes. Flaws in cgroup management can erode security boundaries, as seen in vulnerabilities like CVE-2023-25809, where runc permits unauthorized write access to cgroup hierarchies in rootless containers, and CVE-2019-14891, which involves cri-o's mismanagement of memory cgroups leading to unintended network access. Further highlighting these issues, CVE-2018-20699 demonstrates how Docker can suffer DoS from excessive memory use, and CVE-2014-8171 shows deadlock risks in memory resource controllers, emphasizing the security implications of cgroup misconfiguration.

3.4.2 System Call Exposure $\mathcal{A}_7$. In container security, system calls can source serious vulnerabilities, such as unauthorized access, data leakage, or complete control of host systems and containers. System calls bridge user-space applications and kernel-space functions, offering essential services such as file access, network communication, and process control. Yet, unsecured or mismanaged system calls can serve malicious purposes. This article looks at two exploits related to

such attacks, which show the intricate nature of system call security and the need to consider kernel subsystem design and parameter handling to reduce the risks with system call vulnerabilities.

System Calls Mishandling $\mathcal{E}_{35}$ encompasses vulnerabilities in containerized environments due to flawed system call handling. These calls are vital for kernel-application communication and, if mismanaged, can lead to unauthorized access, privilege escalation, **denial of service (DoS)**, or sensitive data leakage. Examples include CVE-2017-6074, where `DCCP_PKT_REQUEST` mishandling leads to privilege escalation; CVE-2022-0847, with vulnerabilities from uninitialized pipe buffer flags; and CVE-2016-5195 ("Dirty COW"), exploiting a race condition for unauthorized write access. Other CVEs, such as CVE-2014-0038, CVE-2014-4699, and CVE-2017-10661, highlight issues such as incorrect data structure initialization/validation and improper synchronization, emphasizing the need for rigorous system call management to secure container interactions with the kernel.

Syscall Misimplementations $\mathcal{E}_{36}$ refer to vulnerabilities stemming from the flawed implementation of system calls in containerized environments, leading to a range of security breaches such as unauthorized access, privilege escalation, kernel memory overwrite, and sandbox escape. These flaws compromise the crucial isolation in containers, allowing attackers to access host system resources or escalate privileges. Key issues include improper validation and data handling, as demonstrated by CVE-2017-18344 (`timer_create`) and CVE-2017-5123 (`waitid`), where insufficient input validation facilitated outbound access and restricted environment escape. Mishandling of parameters, evident in Polkit's CVE-2021-4034 (`pkexec`), enabled ACE via environment variable manipulation. Memory management oversights, such as in CVE-2018-18281 (`mremap()`) and CVE-2022-42703, resulted in unauthorized memory access. Additionally, CVE-2013-1858 showcases how mismanagement of flag combinations in system calls, such as `CLONE_NEWUSER` and `CLONE_FS`, can lead to privilege escalation, highlighting the nature of syscall misimplementation vulnerabilities.

3.4.3 Virtual File-system $\mathcal{A}_8$. $\mathcal{A}_8$ represents security risks in managing VFSs, such as `procfs`. These attacks exploit improper handling of FDs, permissions, and paths, leading to outcomes such as unauthorized privilege elevation, DoS, information leakage, or host system control.

File System Mismanagement $\mathcal{E}_{37}$ highlights the security vulnerabilities arising from improper management of VFSs, especially `procfs`. Issues stem from inadequate handling of FDs, permissions, and process information. For instance, CVE-2016-1583 exposes how recursive page faults in `procfs` can be exploited for privilege escalation or to trigger a DoS by consuming excessive stack memory. Similarly, CVE-2019-5736 illustrates the risks of flawed FD management, enabling attackers to overwrite the host's runc binary for root access. These examples underline the need for diligent management and comprehensive error checking in VFSs to mitigate security vulnerabilities.

Path Mishandling $\mathcal{E}_{38}$ exposes vulnerabilities in `procfs` within containers, posing risks. CVE-2015-3630 exploits `/proc` path issues, allowing unauthorized access to sensitive data. CVE-2018-10892 permits hardware manipulation by not blocking certain `/proc` paths. CVE-2017-16539 grants unrestricted access to `/proc/scsi`, risking SCSI device manipulation and data loss. CVE-2015-3631 enables policy bypass, including LSM and SELinux rules, leading to privilege escalation highlighting the need for access controls and permissions in managing VFSs like `/proc` to prevent such exploits.

3.4.4 eBPF $\mathcal{A}_9$. eBPF operates as a VM within the kernel, enabling developers to execute bytecode for advanced tracing, monitoring, and functionality extension without modifying the kernel source. In containerized setups, it facilitates load balancing, network policy enforcement, and system observability by tracking kernel and user-space activities. However, its capacity for kernel-space bytecode execution introduces security risks. For instance, exploitable features such as `bpf_probe_write_user` can be abused to manipulate network packets, access or modify other

processes' memory, and alter system call arguments or return codes. Additionally, eBPF's cross-container tracing can be leveraged for attacks across containers in platforms supporting eBPF [68]. These vulnerabilities, stemming from validation inaccuracies or interpreter flaws, include exploits related to register manipulation, boundary violations, and file descriptor mismanagement.

Registers Manipulation $\mathcal{E}_{39}$ in eBPF involves using 32-bit and 64-bit registers, where improper handling can pose security risks. Crossing register boundaries or incorrect sign extensions may lead to out-of-bounds memory access and privilege escalation. CVE-2020-8835, CVE-2021-3490, and CVE-2017-16995 highlight vulnerabilities from boundary and sign extension errors. Proper management is crucial to prevent security breaches.

Boundary Conditions $\mathcal{E}_{40}$ highlights the role of the eBPF VM's verifier in ensuring kernel security by detecting unsafe operations in eBPF programs. The verifier's vulnerability to overlooking arithmetic underflows, overflows, logic errors like incorrect bitwise operations, and inadequate boundary checks poses risks despite its importance. Instances such as CVE-2016-2383, CVE-2020-8835, CVE-2021-3490, and CVE-2017-9150 illustrate how such oversights can lead to control flow disruptions, memory corruption, and improper sign extension, threatening system security with potential outcomes such as data exposure, unauthorized modifications, or ACE. These cases underscore the need for meticulous verification of the boundary conditions in eBPF to prevent exploitation.

eBPF FD Mismanagement $\mathcal{E}_{41}$ poses security risks, including unauthorized privilege escalation and performance degradation. For example, CVE-2016-4557 highlights a flaw in the kernel's eBPF verifier function that failed to manage FD data structures. This oversight allowed local users to manipulate BPF instructions to target incorrect FDs, leading to unauthorized privileges or DoS.

3.4.5 Kernel Modules $\mathcal{A}_{10}$*.* Kernel modules, including device drivers, extend kernel functionality by allowing loading code into the kernel for hardware interaction and other enhanced capabilities. These modules operate with kernel-level privileges, a necessary feature for their functions but one that introduces security vulnerabilities. Containers share the host kernel; this privilege equivalency increases risk, enabling any container to interact with all kernel modules on the host.

Device Driver Manipulation $\mathcal{E}_{42}$ highlights the critical role of device drivers in facilitating kernel-to-hardware communication, crucial for system efficiency and preventing kernel bloat. However, the interaction between containers and these drivers poses notable security risks in containerized environments with a shared kernel, especially in multitenant setups. The high privilege level required for drivers to interact with hardware can be compromised through vulnerabilities accessible to containers, allowing privileged ACE. Examples of such vulnerabilities include Marvell Wi-Fi Chip Driver (CVE-2019-14814), VIVID Driver (CVE-2019-18683), ALSA Timer Driver (CVE-2017-1000380), Host Controller Interface (CVE-2021-3573), and MIDI Driver (CVE-2020-27786). These vulnerabilities, often arising from race conditions, buffer overflows, and use-after-free errors, can lead to system instability, DoS, or ACE, thus compromising system security.

Memory Side-channels $\mathcal{E}_{43}$ Containers run on a shared OS kernel, employing the same memory management subsystem. $\mathcal{E}_{43}$ has identified side-channel exploits in this shared construct, which target the security measures of cache **memory and memory management units (MMUs)** such as ASLR and KASLR. Although these mechanisms introduce randomness and diversify the memory layout of a system, $\mathcal{E}_{43}$ has revealed various methods that can undermine these defenses. Timing attacks, as discussed in References [69, 70], measure the duration of certain memory operations, which can reveal information about the layout of the kernel space, even within containers [71–73]. This highlights the vulnerabilities in kernel space ASLR. Additionally, pre-fetch vulnerabilities, as described in Reference [74], allow attackers to manipulate pre-fetch operations, causing the kernel to access privileged memory details, thus compromising security measures such as

Supervisor Mode Access Prevention (SMAP) and **Supervisor Mode Execution Protection (SMEP)**. Cache-based attacks, as described in Reference [75], allow attackers to observe cache behavior, particularly in shared cache scenarios, to deduce memory layout specifics and bypass protective measures such as ASLR. Furthermore, attackers can read specific kernel memory locations, uncovering data such as KASLR offsets, which can be used to disable KASLR and leave kernel sections vulnerable. Examples of such vulnerabilities include the `/proc/pid/syscall` exploit (CVE-2020-28588) and the *EntryBleed* side-channel attack [76], which depends on the TLB timing (CVE-2022-4543). Some vulnerabilities, such as CVE-2019-10639, even provide attackers with a direct way to bypass KASLR by manipulating connectionless protocols such as UDP or ICMP or initiating IP ID hash collisions to disclose the kernel image offset. Moreover, References [77–82] show that security techniques such as the Linux IMA cloud also introduce side-channel exploits due to their shared internal data structure in a multi-tenant environment.

3.5 Hardware Layer $\mathcal{L}_5$

$\mathcal{L}_5$, encompassing critical components such as CPUs and memory, is susceptible to vulnerabilities arising from hardware optimizations such as speculative execution. These vulnerabilities pose risks including information leakage and ACE.

3.5.1 Hardware-based $\mathcal{A}_{11}$. $\mathcal{A}11$ refers to security breaches stemming from vulnerabilities in hardware design and implementation, particularly those exploited through improper handling of hardware features. We focus on exploits $\mathcal{E}44$ to $\mathcal{E}_{47}$.

Speculative Execution $\mathcal{E}_{44}$ leverages the predictive capabilities of modern processors, which execute instructions speculatively to enhance performance [83]. While this improves efficiency, speculative execution can inadvertently expose sensitive data, forming the basis of side-channel attacks exploiting timing [69] and power consumption [5, 84, 85]. Significant threats include branch prediction vulnerabilities such as Spectre variants (CVE-2017-5753, CVE-2017-5715) [39, 86], which enable unauthorized memory access and control flow hijacking, and L1 data cache vulnerabilities such as CVE-2018-3639 and CVE-2018-3693 [87], allowing attackers to bypass security measures. Meltdown vulnerabilities (CVE-2017-5754, CVE-2018-3640, CVE-2018-3665) reveal sensitive information across kernels, user spaces, and hypervisors [84, 88]. Recent exploits such as Retbleed (CVE-2022-29901, CVE-2022-29900) and Zenbleed (CVE-2023-20593) further demonstrate the evolving nature of speculative execution threats [76, 89, 90]. The intersection of eBPF with speculative execution vulnerabilities, as seen in CVE-2021-31829, CVE-2019-7308, and CVE-2020-13844 [40], highlights the broad impact on computing platforms. These vulnerabilities underscore the challenge of balancing performance with robust security.

Cache Attacks $\mathcal{E}_{45}$ exploit shared CPU cache memory in multi-tenant environments. Side-channel attacks leveraging cache access patterns can reveal information across containers or VMs on the same hardware. Vulnerabilities like Foreshadow (CVE-2018-3620, CVE-2018-3646) demonstrate risks to data stored in the L1 cache, allowing attackers to bypass speculative execution and address translation safeguards to access confidential information, including data in SGX enclaves [91–93]. These highlight the need for isolation and mitigations against cache-based side-channel attacks.

Microarchitectural Data Sampling (MDS) $\mathcal{E}_{46}$ represents sophisticated threats exploiting data remnants in CPU microarchitectural components like buffers, designed to speed up computations but providing vectors for attacks [94]. Vulnerabilities such as Fallout (CVE-2018-12126) [95] and RIDL (CVE-2018-12127, CVE-2018-12130, CVE-2019-11091) [96] target storage buffers, load ports, and fill buffers, allowing unauthorized data access through side channels. Zombieload (CVE-2019-11091) targets uncacheable memory areas, posing risks of data exposure [68]. These

emphasize the challenges in securing CPUs against MDS attacks in environments requiring strict data isolation.

Power Analysis and Timing Attacks $\mathcal{E}_{47}$ involve exploiting variations in power consumption and execution timing to extract sensitive information. Hertzbleed is a recent side-channel attack exploiting dynamic frequency scaling in CPUs to bypass cryptographic protections against timing analysis [5, 69]. Similar vulnerabilities in AMD CPUs, akin to Intel's Meltdown, reveal risks from prefetch instructions leaking virtual address information. Attacks like PLATYPUS exploit the RAPL interface to perform power side-channel attacks, extracting cryptographic keys [97]. Addressing these vulnerabilities requires comprehensive measures, including Page Table Isolation, prefetch configuration adjustments, and restricted access to specific CPU features. Ensuring hardware security in multi-tenant cloud environments remains a persistent challenge, highlighting the balance between performance and security.

4 Container Mitigation Strategies

Transitioning from our taxonomy into practical applications, this section bridges the understanding of attack vectors with actionable security responses. By mapping identified vulnerabilities to corresponding exploits and threats, we introduce strategies $\mathcal{S}$ and mitigation measures $\mathcal{M}$ to address these threats. This framework aims to enhance container security comprehensively, empowering practitioners with the tools needed to safeguard their container environments. Refer to Table 3 for a detailed overview of these mitigation strategies and their associated exploits.

4.1 Software-based Measures

We detail six strategic approaches, including multiple mitigation tactics ranging from $\mathcal{M}1$ to $\mathcal{M}29$.

4.1.1 Container Security and Integrity $\mathcal{S}_1$. This subsection explores strategies to enhance container security, including improving file system security, verifying image authenticity, and implementing secure configurations to safeguard containers against various threats.

Host and Container File System $\mathcal{M}_1$ A critical aspect of protecting the host and container systems is the security of the file system against unauthorized access and potential security breaches. Implementing specific strategies enhances this protection. First, minimizing the attack surface through the judicious use of the copy command in Dockerfiles, which limits file inclusion to essential data only, helps exclude unnecessary data, and thus reduces security vulnerabilities. Second, the restriction of host path volume usage prevents containers from accessing sensitive areas of the host's filesystem, a vital step in mitigating the risks of privilege escalation and data breaches. These strategies counteract exploits, such as $\mathcal{E}_1$ and $\mathcal{E}_4$, which exploit file system vulnerabilities to misuse file permissions, volumes, and binary execution, with the aim of breaching container isolation or compromise the host system. Implementing secure configuration practices, including careful file inclusion and restricting hostPath volumes, prevents unauthorized access and ACE.

Host and Container File System $\mathcal{M}1$ Securing the file system is crucial to prevent unauthorized access and potential breaches. Minimizing the attack surface by limiting file inclusion in Dockerfiles to essential data reduces vulnerabilities. Restricting host path volume usage prevents containers from accessing sensitive host filesystem areas, mitigating risks of privilege escalation and data breaches. These strategies counter exploits such as $\mathcal{E}1$ and $\mathcal{E}_4$, which target file system.

Binary Integrity and Allow-listing $\mathcal{M}_2$ Ensuring binary integrity is key for preventing unauthorized access and ACE within containers. Binary integrity verification measurement techniques (e.g., hash functions) ensure no alterations to the binary's pre-execution. Concurrently, binary allow-listing confines the execution to a predefined list of approved binaries, preventing the run of unauthorized or harmful code. These strategies mitigate the threat $\mathcal{E}_1$, using only authenticated

Table 3. Overview of Security Mitigations $\mathcal{M}$ and Exploit $\mathcal{E}$ Descriptions across Various System Components

$\mathcal{S}$	$\mathcal{M}$	$\mathcal{M}$**itigation Description**	$\mathcal{E}$	$\mathcal{E}$**xploit Description**
$\mathcal{S}_1$	$\mathcal{M}_1$	Use of the copy command judiciously and restriction of hostPath volumes for file system security.	$\mathcal{E}_1, \mathcal{E}_4, \mathcal{E}_{42}$	Addresses Improper Handling of Malicious Binary Exploits and File Manipulations Exploits related to file system operations.
	$\mathcal{M}_2$	Binary integrity verification through hashing and binary allow-listing.	$\mathcal{E}_1, \mathcal{E}_{42}$	Counters Improper Handling of Malicious Binary Exploits by ensuring only authorized binaries execute.
	$\mathcal{M}_3$	Designating file systems as read-only and emphasizing immutability.	$\mathcal{E}_1, \mathcal{E}_4, \mathcal{E}_{22}, \mathcal{E}_{29}$	Mitigates a spectrum of threats by enforcing file system integrity.
	$\mathcal{M}_4$	Enforcing strict file permission settings and protecting against symlink exploits.	$\mathcal{E}_4, \mathcal{E}_9, \mathcal{E}_{27}, \mathcal{E}_{28}, \mathcal{E}_{30}, \mathcal{E}_{31}$	Targets File Manipulations Exploits and Improper File Handling through file permission and symlink protection.
	$\mathcal{M}_5$	Regular scanning and updating of container images and performing vulnerability scans.	$\mathcal{E}_{13}, \mathcal{E}_{14}, \mathcal{E}_{15}$	Addresses vulnerabilities in container images by ensuring they are up-to-date and free from known vulnerabilities.
	$\mathcal{M}_6$	Utilizing digital signatures for image source integrity and enhancing image manifest verification protocols.	$\mathcal{E}_{14}, \mathcal{E}_{20}, \mathcal{E}_{42}$	Mitigates Image Spoofing by verifying the source and image integrity.
	$\mathcal{M}_7$	Selecting base images from reputable sources to ensure integrity and authenticity.	$\mathcal{E}_{13}, \mathcal{E}_{14}, \mathcal{E}_{42}$	Directly addresses vulnerabilities exploited by Base Image Propagation and Image Spoofing Exploits.
	$\mathcal{M}_8$	Imposing secure configurations on images and sanitizing Git URLs.	$\mathcal{E}_{13}$, $\mathcal{E}_{15}$, $\mathcal{E}_{20}$, $\mathcal{E}_{42}$	Essential for mitigating risks associated with Base Image Propagation and Build Manipulation Exploits through secure handling.
$\mathcal{S}_2$	$\mathcal{M}_9$	Adoption of advanced runtime security measures, including RASP and secure container runtimes like gVisor.	$\mathcal{E}_9$, $\mathcal{E}_{10}$, $\mathcal{E}_{22}$, $\mathcal{E}_{24}$, $\mathcal{E}_{29}$, $\mathcal{E}_{35}$, $\mathcal{E}_{36}, \mathcal{E}_{39}$	Covers a range of exploits from Improper File Handling to Overflows Exploits, addressing unauthorized access, data leakage, or ACE.
	$\mathcal{M}_{10}$	Utilization of namespaces and cgroups for resource and access management, and enforcing least privilege through read-only configurations.	$\mathcal{E}_{23}$, $\mathcal{E}_{34}$, $\mathcal{E}_{37}$, $\mathcal{E}_{38}$	Covers a range of exploits from Improper File Handling to Overflows Exploits, addressing unauthorized access, data leakage, or ACE.
	$\mathcal{M}_{11}$	Installation of runtime monitoring tools and strengthening of API server access controls for enhanced monitoring and management.	$\mathcal{E}_{17}$, $\mathcal{E}_{18}$, $\mathcal{E}_{35}$, $\mathcal{E}_{36}$	Targets Security Policy Mishandling and Privilege Mismanagement Exploits, arising from inadequate configuration and policy management.
	$\mathcal{M}_{12}$	Ensuring the security of Unix socket communications between containers to prevent data tampering and eavesdropping.	$\mathcal{E}_{18}, \mathcal{E}_{33}$	Focuses on Privilege Mismanagement Exploits, particularly stemming from misconfigurations and security flaws related to Unix socket communications.
$\mathcal{S}_3$	$\mathcal{M}_{13}$	Implementation and enforcement of Role-Based Access Control (RBAC) for Kubernetes API access.	$\mathcal{E}_2, \mathcal{E}_5, \mathcal{E}_{32}$	Targets vulnerabilities leading to Control Plane Bypassing and Unauthorized Access and Secrets Exploits, regulating API access to prevent unauthorized control or data access.
	$\mathcal{M}_{14}$	Strengthening API server authentication and authorization mechanisms.	$\mathcal{E}_2, \mathcal{E}_{32}$	Addresses Control Plane Bypassing Exploits by securing API interactions against unauthorized access and privilege escalation.
	$\mathcal{M}_{15}$	Implementing stringent admission control mechanisms, such as PodSecurityPolicy.	$\mathcal{E}_2, \mathcal{E}_4, \mathcal{E}_{16}, \mathcal{E}_{30}, \mathcal{E}_{32}$	Mitigates Control Plane Bypassing and File Manipulations Exploits through enforcement of secure workload and user behaviors.
	$\mathcal{M}_{16}$	Employing enhanced secrets management practices for superior encryption of Kubernetes Secret objects.	$\mathcal{E}_5, \mathcal{E}_{16}$	Prevents Unauthorized Access and Secrets Exploits by securing sensitive information against unauthorized disclosure.
	$\mathcal{M}_{17}$	Implementation of Kubernetes network policies and fine-grained network controls.	$\mathcal{E}_5, \mathcal{E}_7$	Defends against Unauthorized Access and Secrets as well as Traffic Manipulation and Redirection Exploits, ensuring secure pod communications and network traffic integrity.
$\mathcal{S}_4$	$\mathcal{M}_{18}$	Establishment of robust encryption practices including mTLS, HTTPS, and HSTS for secure network communications.	$\mathcal{E}_6, \mathcal{E}_7, \mathcal{E}_{16}$	Targets Overlay Network Encryption, Traffic Manipulation and Redirection, and Registry Mishandling Exploits, enhancing data confidentiality and integrity.
	$\mathcal{M}_{19}$	Network segmentation and traffic management through service meshes and IDS/IPS, promoting secure communication and policy enforcement.	$\mathcal{E}_6, \mathcal{E}_7, \mathcal{E}_{26}$	Defends against Overlay Network Encryption, Traffic Manipulation and Redirection, and Network Protocol Exploits by isolating network segments and enhancing security enforcement.
	$\mathcal{M}_{20}$	Implementation of eBPF safeguards, security audits, and DMZs for advanced network security and monitoring.	$\mathcal{E}_{25}$	Addresses Netfilter Subsystem Exploits by enhancing network monitoring, auditing vulnerabilities, and segregating internal network from external threats.
$\mathcal{S}_5$	$\mathcal{M}_{21}$	Implementation of secure coding practices, input validation, safe data handling, and the use of parameterized queries.	$\mathcal{E}_3, \mathcal{E}_8, \mathcal{E}_9, \mathcal{E}_{10}, \mathcal{E}_{11}$	Targets vulnerabilities from Input and Parsing Mishandling to Code Injection and Improper File Handling.
	$\mathcal{M}_{22}$	Adoption of secure configurations and HTTP security headers for enhanced web application security.	$\mathcal{E}_2, \mathcal{E}_{11}, \mathcal{E}_{12}$	Mitigates Control Plane Bypassing and Code Injection Exploits by securing application and infrastructure configurations.
	$\mathcal{M}_{23}$	Deployment of WAFs and integration of RASP tools for real-time attack detection and mitigation.	$\mathcal{E}_3, \mathcal{E}_8, \mathcal{E}_{11}$	Defends against Input and Parsing Mishandling, Input Manipulation, and Code Injection Exploits.
	$\mathcal{M}_{24}$	Enhancing data processing security through rigorous coding standards and input validation.	$\mathcal{E}_3, \mathcal{E}_{42}, \mathcal{E}_{43}$	Addresses Input and Parsing Mishandling Exploits, focusing on secure data handling and validation.
	$\mathcal{M}_{25}$	Comprehensive configuration management, security auditing, and up-to-date security measures for container environments.	$\mathcal{E}_{17}$, $\mathcal{E}_{19}$, $\mathcal{E}_{21}$, $\mathcal{E}_{42}, \mathcal{E}_{43}$	Targets Security Policy Mishandling, Daemon Disruption, and Patch Regression Exploits by ensuring secure configurations and audits.
	$\mathcal{M}_{26}$	Continuous secure coding training, diligent patch management, and dynamic analysis to monitor FD usage.	$\mathcal{E}_{40}$, $\mathcal{E}_{41}$, $\mathcal{E}_{42}$, $\mathcal{E}_{43}$	Addresses Boundary Conditions and eBPF FD Mismanagement Exploits, emphasizing the importance of updates and secure coding practices.
$\mathcal{S}_6$	$\mathcal{M}_{27}$	Regular updates to microcode or BIOS.	$\mathcal{E}_3, \mathcal{E}_8, \mathcal{E}_9, \mathcal{E}_{11}, \mathcal{E}_{44}, \mathcal{E}_{46}$	Mitigates Speculative Execution Exploits by updating hardware vulnerabilities inherent in speculative execution processes.
	$\mathcal{M}_{28}$	Disabling hyperthreading and Simultaneous Multithreading (SMT) to prevent data leakage by cross-threading. Implementing compiler and ACE defenses such as retpoline and memory barriers. Cache and memory management enhancements for dedicated cache segments and isolation.	$\mathcal{E}_{45}, \mathcal{E}_{46}, \mathcal{E}_{44}$	Addresses vulnerabilities exploited by Cache Attacks and Microarchitectural Data Sampling (MDS) Exploits, preventing cross-tenant data exploitation and sensitive data leakage in cache memory.
	$\mathcal{M}_{29}$	Reducing timer precision and constraining access to RAPL interface for mitigating timing and power measurement attacks.	$\mathcal{E}_{45}, \mathcal{E}_{47}$	Targets techniques employed in cache attacks and power analysis and timing exploits, reducing the risk of sensitive data inference through timing and power analysis.

executable binaries. These measures reduce the risk of incidents, such as unauthorized file access or system compromise, by obstructing the introduction and execution of compromised binaries.

File System Security $\mathcal{M}_3$ Implementing robust file system security measures within containers is essential to block unauthorized modifications and maintain application integrity. Setting file systems to read-only limits write access to only necessary areas, diminishing the possibility of malicious changes. Additionally, adopting immutability for mounted container file systems prevents alterations, further securing the integrity of the application. These approaches are vital

in protecting against threats that exploit vulnerabilities such as unauthorized file system access and manipulation, as seen in exploits $\mathcal{E}_1$, $\mathcal{E}_4$, $\mathcal{E}_{22}$, and $\mathcal{E}_{29}$. These vulnerabilities exploit inadequate validation and defined trust boundaries to compromise container and host. By enforcing read-only access and immutability, it thwarts unauthorized changes.

File Permission and Symlink Protection $\mathcal{M}_4$ Proper configuration of file permissions and protection against symlink attacks are fundamental for container security. Setting stringent file permissions limits access to files and directories, reducing the risk of unauthorized use and measures to thwart symlink attacks and prevent unauthorized file access or alterations. These safeguards are essential to address the vulnerabilities targeted by $\mathcal{E}_4$ and $\mathcal{E}_9$, which exploit weaknesses in file access and modification. Implementing robust file permission policies and anti-symlink attack mechanisms bolsters container security, mitigating the risks of data breaches and unauthorized changes.

Image Scanning and Updating $\mathcal{M}_5$ Scanning and updating of images are critical for ensuring security and thwarting exploitation attempts. These processes help organizations keep their images free from known vulnerabilities and flaws. Using specialized tools for vulnerability scanning offers evaluation of container images, identifying potential security weaknesses. Such practices are vital in countering vulnerabilities targeted by $\mathcal{E}_{13}$, $\mathcal{E}_{14}$, and $\mathcal{E}_{15}$, which exploit outdated or compromised images. By committing to ongoing image scanning and updates with advanced detection tools, organizations can reduce the risk of ACE and system compromises, strengthening container security against threats such as image spoofing, base image propagation, and build manipulation.

Source and Content Verification $\mathcal{M}_6$ Ensuring the authenticity container images through source and integrity verification is critical. Employing digital signatures helps to confirm that the images are unmodified from their original version, preserving their integrity. Strengthening image manifest verification further ensures that images come from reputable sources. Such practices are vital to defend against exploits such as $\mathcal{E}_{14}$, which target image identification and validation vulnerabilities due to repository spoofing, path traversal, and cache poisoning. Implementing strict verification protocols, including digital signatures and manifest checks, is crucial to protecting container environments from spoofing attacks and maintaining the integrity and authenticity of images.

Trusted Source $\mathcal{M}_7$ Choosing containers from trusted sources is crucial to avoid the use of compromised images and mitigate the risk of security breaches. This critical strategy is especially relevant to protect against the vulnerabilities targeted by $\mathcal{E}_{13}$ and $\mathcal{E}_{14}$, which highlight the risks associated with an unsecured source and insufficient validation of container images. By committing to reliable and verified sources, organizations can preserve the integrity and authenticity of their base images, thus preventing exploitation through vulnerabilities and spoofing techniques.

Secure Configuration $\mathcal{M}_8$ Adopting secure configurations and handling practices is critical to cut vulnerabilities within container images. Enforcing secure settings by turning off unneeded services or features reduces the number of attack surfaces. Sanitizing Git URLs is another step to prevent redirection and code injection attacks by validating URLs. These steps address the vulnerabilities highlighted by $\mathcal{E}_{13}$ and $\mathcal{E}_{15}$, underscoring the importance of secure image sourcing, configuration, and handling. Through secure configurations and rigorous source verification, including Git URL sanitization, these limit attack vectors and protect against ACE and system compromises.

4.1.2 Runtime Security and Isolation $\mathcal{S}_2$. Runtime Security and Isolation is pivotal in maintaining a secure containerized environment, focusing on preventing unauthorized access and ensuring strict separation between containerized applications during execution, entailing a multifaceted approach, incorporating advanced technological solutions and strategic configurations to mitigate risks.

Container Runtime Security $\mathcal{M}_9$ Implementing advanced runtime security measures, such as **Runtime Application Self-Protection (RASP)** and secure container runtimes such as gVisor, is crucial to protect container environments. RASP improves security by offering real-time threat detection and mitigation, while gVisor and similar technologies improve isolation and reduce breaches risks. Together, these measures address vulnerabilities prone to unauthorized access, data leaks, or ACE, as seen in $\mathcal{E}_9$, $\mathcal{E}_{22}$, $\mathcal{E}_{24}$, $\mathcal{E}_{29}$, and $\mathcal{E}_{10}$. By leveraging such runtime security solutions, organizations can create more sandbox containers, enhancing the security of container execution environments.

Resource and Access Management $\mathcal{M}_{10}$ Leveraging namespaces and cgroups is essential for managing resources and access in container environments, restricting containers to only their allotted resources, thus reducing the scope of security breaches. Implementing extra security measures, such as removing unnecessary capabilities and setting file systems to read-only, embodies the principle of least privilege, enhancing protection. These strategies address vulnerabilities targeted by $\mathcal{E}_{23}$, $\mathcal{E}_{34}$, $\mathcal{E}_{37}$, and $\mathcal{E}_{38}$. Through resource and access controls, including isolation and resource allocation, these practices strengthen security against a range of vulnerabilities.

Runtime Security Monitoring and Management $\mathcal{M}_{11}$ Runtime Security Monitoring and Management are crucial to maintaining the oversight and control of container activities. Implementing monitoring tools allows organizations to observe container behavior, facilitating the immediate identification of anomalies and threats. Enhancing access controls on the container orchestrator's API server further ensures that interactions with the orchestration layer are limited to authorized personnel, protecting against unauthorized access and activities. These practices address vulnerabilities related to misconfigurations and mismanagement of security policies, as seen in $\mathcal{E}_{17}$ and $\mathcal{E}_{18}$. By combining precise configuration, rigorous policy enforcement, and advanced security technologies, organizations can protect against risks arising from runtime and policy vulnerabilities, underscoring the importance of comprehensive security in containers.

Secure Inter-container Communications $\mathcal{M}_{12}$ Ensuring the security of inter-container communications is vital to block lateral movement and strengthen security in containerized environments. Securing Unix socket communications prevents eavesdropping and data tampering, addressing vulnerabilities such as those seen in $\mathcal{E}_{18}$, which arise from misconfigurations and security flaws, particularly in Unix socket access controls. These vulnerabilities can ease unauthorized access, container escape, and ACE. Implementing robust security measures for container run-time configurations is crucial to maintaining the integrity and confidentiality of container communications.

4.1.3 Orchestration Enhancements $\mathcal{S}_3$. In the landscape of container orchestration, Kubernetes emerges as the de facto standard, necessitating robust security enhancements to protect against threats. Implementing these security enhancements involves various mitigation strategies to protect Kubernetes environments against unauthorized access, data breaches, and other vulnerabilities.

Role-Based Access Control (RBAC) $\mathcal{M}_{13}$ Implementing RBAC is crucial for securing Kubernetes by regulating access to its API. RBAC ensures that individuals and services have only the essential permissions for their functions, reducing the potential for unauthorized actions and decreasing the attack surface. It counters vulnerabilities associated with orchestration control plane exploits, such as $\mathcal{E}_2$ and $\mathcal{E}_5$, which target unauthorized control and data access through misconfigurations of the control plane or software flaws. Through the precise deployment of RBAC, organizations can reduce these risks, fostering a secure Kubernetes ecosystem.

API Security $\mathcal{M}_{14}$ Enhancing the security of the API server through authentication and authorization is critical to protecting orchestration environments. Implementing security mechanisms, including **mutual TLS (mTLS)** authentication, OAuth2 [98], and integration with identity providers,

is essential to verify entities and tool strict access controls. These practices counteract the vulnerabilities exploited by $\mathcal{E}_2$, which arise from weaknesses in the orchestration control plane, particularly in the API server and admission controllers. Mitigating unauthorized access and privilege escalation includes improving authentication and authorization, minimizing the exploits risks.

Authentication Security $\mathcal{M}_{15}$ Using admission control mechanisms such as PodSecurityPolicy is critical for securing Kubernetes by establishing secure defaults for workloads and users. By preventing the execution of unauthorized or malicious pods, these policies enforce security best practices, such as limiting root access and restricting the use of privileged containers, thus narrowing the attack surface. Such precautions are crucial for guarding against vulnerabilities targeted by $\mathcal{E}_2$ and $\mathcal{E}_4$, which exploit security configuration oversights and inadequate best practice enforcement to achieve unauthorized access and privilege escalation. Through PodSecurityPolicy, organizations can improve security by controlling pod operations and file management, reducing the risks.

Key and Secrets Managements $\mathcal{M}_{16}$ Implementing robust secrets management practices is crucial to secure sensitive data within Kubernetes environments. Using Kubernetes Secret objects, enhanced with extra encryption measures, or integrating specialized secrets management tools, organizations can improve the encryption of secrets such as passwords, tokens, and SSH keys. This comprehensive approach ensures the protection of sensitive information at rest and in transit, addressing vulnerabilities that lead to $\mathcal{E}_5$. Such exploits result from security misconfigurations or software defects, exposing sensitive data. The risk of unauthorized access to sensitive information is reduced through advanced $\mathcal{M}_{16}$, which improves data security in orchestration environments.

Networking Policy $\mathcal{M}_{17}$ Implementing fine-grained Kubernetes network policies is crucial for managing pod communications within and across clusters, a key strategy in protecting network traffic and preventing unauthorized data exchanges. These policies reduce the risk of network-based attacks by enabling precise control over pod interactions and isolating sensitive workloads. Vulnerabilities targeted by $\mathcal{E}_5$ and $\mathcal{E}_7$, which exploit containers orchestration framework weaknesses to enable unauthorized access or manipulate network traffic, can be countered with strict network policy enforcement. This approach strengthens the integrity and security of the network infrastructure, protecting against potential breaches and unauthorized activities.

4.1.4 Network Encryption and Security Mitigation Strategies $\mathcal{S}_4$*.* Adopting comprehensive network encryption and security mitigations is crucial to protect data in transit, cut attack surfaces, and provide access to resources. This section outlines a multifaceted approach toward achieving network security through encryption practices, segmentation, management, and network safeguards.

Encryption Practices $\mathcal{M}_{18}$ Implementing comprehensive encryption practices is essential to safeguard network security, including validating encryption protocols for overlay networks, establishing end-to-end encryption, and using mTLS to secure all intra-cluster communications. The deployment ensures data integrity and confidentiality. Adopting comprehensive network encryption and security mitigation strategies is crucial to protecting data in transit, minimizing attack surfaces, and securing browser connections. Such practices counteract the vulnerabilities targeted by $\mathcal{E}_6$, $\mathcal{E}_7$, and $\mathcal{E}_{16}$, bolstering data confidentiality, integrity, and availability. Organizations can protect themselves against unauthorized access, data manipulation, and extra security threats by adopting thorough encryption methodologies, secure configurations, and permissions management.

Segmentation and Traffic Management $\mathcal{M}_{19}$ Network segmentation is critical for reducing the attack surface, isolating segments, and limiting access to essential services, thus containing potential threats. Service meshes such as Istio or Linkerd help with precise security enforcement and traffic management, allowing for detailed control over service communications and policy

implementation. Integrating **Intrusion Detection Systems (IDS)** and **Intrusion Prevention Systems (IPS)**, along with protocol hardening and strict segmentation, strengthens network defenses by identifying and blocking unauthorized access while maintaining adherence to security protocols. These approaches are vital for addressing vulnerabilities targeted by $\mathcal{E}_6$, $\mathcal{E}_7$, and $\mathcal{E}_{26}$, which capitalize on security misconfigurations, insufficient practices, and protocol weaknesses to conduct unauthorized access, data breaches, and disrupt services. By adopting a holistic approach to segmentation, implementing secure traffic management, and applying comprehensive security measures, organizations can protect against these threats, improving the security of the container network.

Network Safeguards and Audits $\mathcal{M}_{20}$ Measures such as eBPF safeguards, regular security audits, and the deployment of DMZ are vital to strengthening network defenses. eBPF facilitates high-performance security monitoring and analysis, offering kernel-level tracking and policy enforcement. Security audits are critical to identifying vulnerabilities and verifying adherence to best practices. DMZs enhance security by isolating the internal network from external access and providing a buffer zone that protects internal resources. These strategies address vulnerabilities such as those in $\mathcal{E}_{25}$, which involve the Netfilter kernel module and can result in unauthorized access or system disruptions. Through the strategic use of eBPF and security audits, organizations can improve network resilience, reducing exploit risks.

4.1.5 Secure Coding and Application Design $\mathcal{S}_5$. Implementing secure coding practices and thoughtful application design are the foundations of a secure application. This section outlines the methodologies and best practices for software development, emphasizing the importance of mitigating vulnerabilities from the initial stages of development through deployment and maintenance.

Secure Coding $\mathcal{M}_{21}$ Secure coding practices is key for containerized applications. This includes following solid coding principles, validating and sanitizing inputs, securing data handling, protecting scripting environments, and using parameterized queries, mitigating $\mathcal{E}3$, $\mathcal{E}8$, $\mathcal{E}9$, and $\mathcal{E}_{11}$.

Security Policies and Configurations $\mathcal{M}_{22}$ Enforcing secure configurations and policies reduces the attack surface. Enforcing HTTP security headers such as **Content Security Policy (CSP)**, X-Content-Type-Options, and X-Frame-Options counters content injection risks and bolsters web security, defending against exploits such as $\mathcal{E}2$ and $\mathcal{E}_{11}$.

Web Application Firewalls (WAFs) *and Runtime Protection* $\mathcal{M}_{23}$ Deploying WAFs and RASP tools strengthens web application defenses. WAFs filter out suspicious requests, while RASP analyzes behavior and traffic to detect and neutralize threats in real-time, protecting against vulnerabilities targeted by exploits such as $\mathcal{E}3$, $\mathcal{E}8$, and $\mathcal{E}11$.

Data Processing Security $\mathcal{M}_{24}$ Enhancing data processing security is essential for defending against exploits such as $\mathcal{E}_3$. This involves implementing rigorous coding standards, thorough input validation, and secure data handling protocols. Incidents such as CVE-2019-11253 and CVE-2019-11254 show the critical need to confirm and sanitize YAML and JSON inputs to prevent DoS attacks and ACE. By prioritizing data integrity and security, organizations can protect themselves more against vulnerabilities, ensuring higher protection against threats to data-processing operations.

Configuration Management and Auditing $\mathcal{M}_{25}$ Utilizing configuration management tools promotes consistent deployments and aids in auditing. Secure daemon configurations, input validation, and the application of safe settings protect daemon processes from exploitation. Continuous updates and strict version control counteracts $\mathcal{E}17$, $\mathcal{E}19$, and $\mathcal{E}21$.

Secure Coding and Patch Management $\mathcal{M}_{26}$ Regular updates with patches defend against new vulnerabilities, while secure coding training prepares developers for security challenges. Dynamic analysis tools for tracking file descriptor usage help detect issues early, countering $\mathcal{E}40$ and $\mathcal{E}_{41}$.

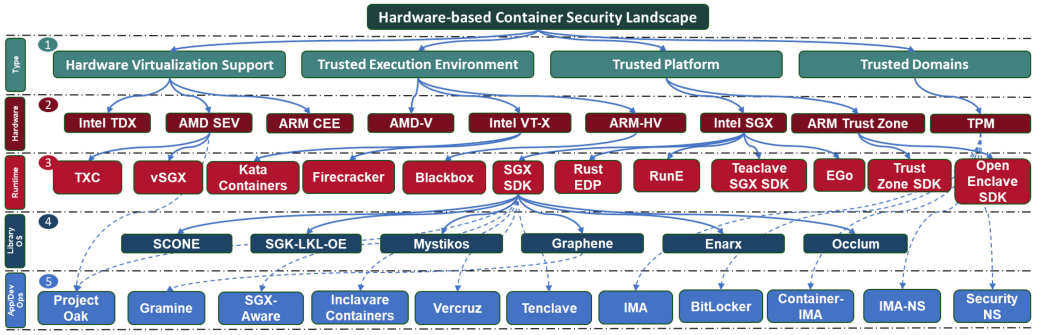

Fig. 4. Landscape hardware-based container security measures.

4.2 Hardware-based Security Measures

The rise of confidential computing, grounded in hardware-based container security, marks a pivotal shift in cloud security. By leveraging hardware features such as improved isolation, encryption, and remote attestation through TEEs, **Hardware Virtualization Support (HVS)**, Trusted Domains, and Trusted Platforms (see Table 3), this framework strengthens data confidentiality, integrity, and authenticity [99, 100]. These strategies enhance container security by utilizing TEEs and advanced hardware components, protecting against threats from malicious insiders, compromised OSs, and external adversaries. However, deploying secure microchips and running code in isolated environments introduces complexity and cost. This section outlines four hardware-based container strategies, categorized by trust assumptions, detailed in Section 6. Figure 4 shows a hierarchical diagram illustrating different types of hardware-based protocols.

Trusted Platforms $\mathcal{H}30$ improve container security by integrating **hardware security modules (HSMs)** such as the **Trusted Platform Module (TPM)**. The TPM utilizes secure boot, disk encryption, and remote attestation to ensure system integrity and manage cryptographic keys. The unique **endorsement key (EK)** of the TPM authenticates the platform's identity, crucial for preventing attacks such as orchestration exploits $\mathcal{A}1$ and application-level threats $\mathcal{A}2$. Linux's **Integrity Measurement Architecture (IMA)** checks file integrity and detects unauthorized changes. Innovations such as Container-IMA [79] and IMA-NS [67] further enhance container integrity verification and isolate security policies, effectively mitigating risks from malicious binary deployment $\mathcal{E}1$ and protecting against $\mathcal{L}3$ and $\mathcal{L}4$ attacks. These measures maintain the integrity of containerized environments from boot to run-time, safeguarding against a spectrum of threats, including input manipulation, code injection, and unauthorized data access $\mathcal{E}8$ to $\mathcal{E}12$.

TEEs $\mathcal{H}_{31}$, exemplified by Intel SGX [101, 102] and Arm's TrustZone [103], play a role in enhancing the security of containerized applications. By isolating code and data from the central OS and potential vulnerabilities, TEEs provide a secure layer for applications within hardware, VMs, or secure enclaves, supported by tools such as SCONE [104] and Mystikos [105]. TEEs address security challenges across $\mathcal{L}$. They mitigate $\mathcal{A}_1$ attacks by isolating execution spaces and protecting against unauthorized access, tampering, malicious binary substitution, and traffic manipulation. TEEs protect against $\mathcal{A}_2$ attacks, ensuring the integrity and confidentiality of code execution, data processing, and storage. This protection extends to protecting against input manipulation $\mathcal{E}_8$, improper file handling $\mathcal{E}_9$, overflows $\mathcal{E}_{10}$, code injection $\mathcal{E}_{11}$, and default misconfigurations $\mathcal{E}_{12}$. TEEs provide hardware-level isolation for $\mathcal{A}_3$ attacks, particularly effective against namespace isolation breaches, ensuring protection from host or container-based attacks, including PID and network namespace vulnerabilities. By leveraging TEEs, containers can achieve enhanced security, enforce access controls, and maintain isolated execution, mitigating vulnerabilities and vectors.

Trusted Domains $\mathcal{H}_{32}$ powered by Intel **Trusted Domain Extensions (TDX)** [101] and AMD **Secure Encryption Virtualization (SEV)** [106], serve as a foundational security measure for containers, addressing vulnerabilities from orchestration to the host layer. Intel TDX and AMD SEV create secure, isolated environments that mitigate security challenges, ensuring integrity and confidentiality against orchestration layer attacks $\mathcal{A}1$ such as unauthorized access and manipulation and exploits such as binary substitution and control plane bypassing. They also defend the application layer $\mathcal{A}2$, protecting against input manipulation $\mathcal{E}8$ and default misconfigurations $\mathcal{E}12$, preventing unauthorized access and execution. TDX and SEV enhance container image integrity in the engine layer, preventing image spoofing and manipulation $\mathcal{A}3$, while protecting the host layer from PID and network namespace attacks $\mathcal{A}_4$. Runtime tools such as TCX [99] and vSGX [107] operationalize these capabilities, enabling confidential computing and reinforcing container security. This integrated security strategy defends against unauthorized manipulation, data breaches, and privilege escalations, meeting modern container environments' manageability and agility needs.

Hardware Virtualization Support $\mathcal{H}_{33}$, utilizing CPU features such as Intel VT-x, AMD-V, and ARM-HV, improves performance and security for VMs and containers in cloud environments. Tools such as nested paging and SR-IOV, along with projects such as Kata Containers [54, 108] and Firecracker [109], leverage KVM to create secure, isolated microVM spaces. However, challenges remain, such as operation forwarding attacks that necessitate further enhancements. Using encrypted memory for isolation using CPU privilege levels without hardware emulation can boost container security across different architectures [110]. Moreover, integrating these virtualization technologies with remote attestation and specialized runtimes can mitigate host layer vulnerabilities, including namespace breaches, thereby strengthening isolation and enhancing trust and adoption in cloud environments.

4.2.1 Hardware Security Enhancements $\mathcal{S}_6$. This explores strategies and mitigations to address hardware and firmware vulnerabilities, protecting systems against hardware-centric attacks such as speculative execution and side-channel exploits.

Hardware and Firmware Security $\mathcal{M}_{27}$ Addressing hardware and firmware vulnerabilities is critical, with regular microcode or BIOS updates to combat $\mathcal{E}_{44}$, which target speculative execution in processors. These processes can leave traces, enabling attackers to use side-channel information to access data. The exploits Spectre and Meltdown underscore the importance of updates in mitigating risks from speculative execution and preventing breaches [84, 86, 88].

Side-channel and Speculative Execution $\mathcal{M}_{28}$ Addressing threats that exploit hardware architecture complexities requires a set of mitigation strategies: (i) *Processor and Multithreading Configuration:* Disabling hyperthreading and **Simultaneous Multithreading (SMT)** reduces cross-thread data leakage risks; (ii) *Compiler and Code Execution Defenses:* Utilizing retpoline during software recompilation guards against branch target injection attacks, such as Spectre Variant 2, and installing compiler mitigations that enforce memory barriers to deter speculative execution attacks. (iii) *Cache and Memory Management:* Allocating dedicated cache segments and incorporating cache flush instructions during cryptographic operations improve isolation and prevent data retention in cache memory, mitigating cache side-channel attack risks. These proactive measures counter vulnerabilities such as $\mathcal{E}_{45}$, $\mathcal{E}_{46}$, and $\mathcal{E}_{47}$, enhancing protection against unauthorized data access and leakage via side-channel and speculative execution pathways. By implementing these strategies, organizations can reduce the likelihood of inference or extraction of sensitive data through shared hardware resources, establishing a defense against complex hardware-based threats.

Timing and Power Measurement $\mathcal{M}_{29}$ Timing and power analysis attacks exploit hardware vulnerabilities to acquire sensitive data illicitly. Effective mitigation strategies include: (i) *Timing Attacks:* Diminishing the precision of timers accessible to containers complicates attackers' ability to

perform execution time analysis, thus reducing the efficacy of timing attacks aimed at deducing system operations or extracting data. (ii) *Power Side-channel Attacks:* Limiting access to the **Running Average Power Limit (RAPL)** interface prevents attackers from observing power consumption patterns, a method used to infer sensitive information. Implementing these measures addresses vulnerabilities targeted by exploits such as $\mathcal{E}_{45}$ and $\mathcal{E}_{47}$, substantially reducing the risk of unauthorized data access or insight through hardware vulnerabilities. These mitigations strengthen defenses against techniques designed to leverage timing and power measurements by decreasing the accuracy of timing assessments and controlling power consumption data.

5 Lesson Learned

Key insights $\mathcal{I}$ serve as the foundation for the integrated framework. Each insight is thoroughly outlined, explained, and justified before integration into the framework's development:

$\mathcal{I}_1$ - **Unikernels for Container Security:** Unikernels, as specialized OS layers, reduce the codebase by compiling only the necessary components required for a specific application, enhancing performance and scalability [111–113]. They improve container security by minimizing the **Trusted Computing Base (TCB)**, which encompasses critical hardware, software, and firmware components. A smaller TCB reduces the attack surface and exposure, as discussed in Section 3. Our analysis highlights vulnerabilities in the engine ($\mathcal{L}_3$) and host ($\mathcal{L}_4$). However, transitioning from a monolithic OS to unikernels presents challenges due to OS optimization complexity and practicality challenges. This work demonstrates the link between TCB components and container vulnerabilities, offering strategies to reduce risks through unikernels.

$\mathcal{I}_2$ - **Cloud Containers Follow a Simple Deployment Model:** Cloud containers typically use a single-user, single-application model in microservice environments. This focused approach tailors each container to specific applications and users, influencing security strategies with protocols optimized for each application's unique requirements. Understanding this model allows for better TCB management and improved overall security.

$\mathcal{I}_4$ - **Linux's Lack of Container Contextualization Opens Security Weaknesses:** In multi-tenant environments, the IMA and other LSMs face challenges due to the kernel's inability to recognize containers as distinct entities, a concept known as *Userspace Fiction* [114]. This limitation hinders container autonomy, increases privacy risks during remote attestation, and complicates integrity checks, undermining IMA's role in securing the kernel [67, 79]. Enhanced with TPM and container image signing, IMA defends against unauthorized access by verifying critical files using cryptographic hashes. However, vulnerabilities in Docker Notary, Docker Engine, and Sylabs Singularity highlight trust verification issues [115]. Efforts to improve IMA's container integration, such as segmentation, virtualization, and namespacing, aim to enhance security but add complexity, potentially leading to inefficiencies and new vulnerabilities [77, 78]. A unified framework integrating enhanced virtualization and namespacing should be developed to optimize IMA in multi-tenant environments, allowing precise controls while maintaining integrity [79].

$\mathcal{I}_5$ - **Confidential Computing Offers Robust Security Enhancement:** Confidential computing improves security through hardware-based encryption and TEEs, which secure data processing and storage in cloud environments. This technology ensures data integrity, privacy, and trust within container ecosystems [99, 116–118]. However, practical deployment and operational flexibility may be challenging. Combining hardware and software security techniques can further enhance container integrity and confidentiality (see Section 4).

$\mathcal{I}_6$ - **Case Study: eBPF's Dual Role in Container Security:** eBPF serves a paradoxical role for containers, offering both security enhancements and potential vulnerabilities. Security tools such as Cilium [119], Calico [120], Hubble, and Falco [121, 122] leverage eBPF to improve visibility, control, and performance of container-based applications. These tools use eBPF for

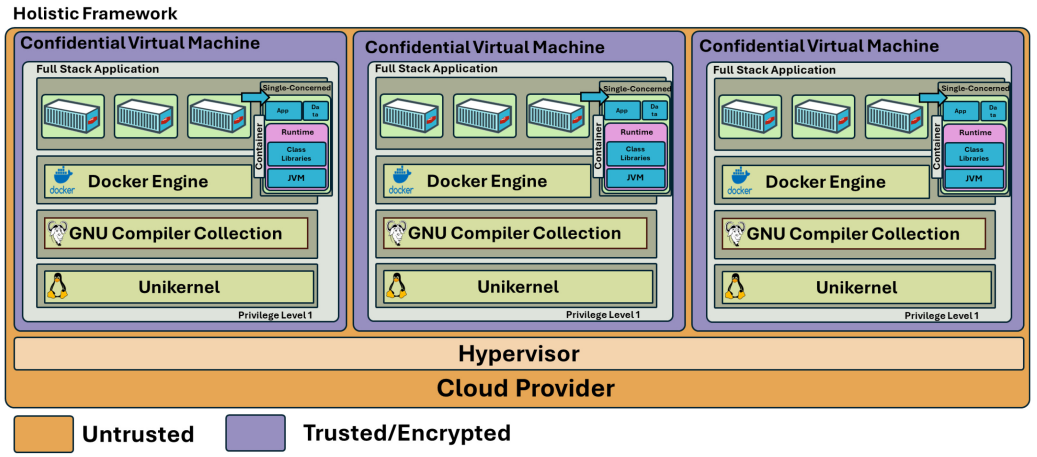

Fig. 5. CVMs deploying Unikernel-based TCB-optimized confidential containers, each encapsulating a full-stack application.

efficient network packet filtering, policy enforcement within the kernel, real-time system call monitoring, and anomaly detection, thereby enhancing container security. However, employing eBPF within kernel space introduces serious security threats (see Section 3.4.4). The most vulnerable environments permit users to execute custom code, highlighting online development platforms and CI/CD platforms. As per Reference [68], about 50% allows potent system calls with enabled eBPF, a recipe for malicious eBPF feature exploitation; over 30% are prone to escape attacks, threatening container isolation. Thus, users must understand the risk of eBPF usage relative to their specific usecase and apply effective mitigations, which include implementing strict access controls on eBPF features, limiting user permissions for package installation, and deploying containers within VMs to improve isolation [123].

6 Holistic Framework and Emerging Technologies

As emerging technologies advance, there is a growing emphasis on granting users greater control over their workloads deployed in the cloud, enhancing confidentiality and trust. To achieve these objectives, we propose an integrated framework, shown in Figure 5, that shields workloads from external threats and ensures internal trust and confidentiality within the TCB to address vulnerabilities. Figure 5 shows our holistic framework: It leverages confidential computing, utilizing hardware-based encryption and TEEs to fortify security by deploying confidential containers. By isolating and encrypting data during processing and storage, confidential computing reduces the need to trust cloud providers, thereby mitigating risks of unauthorized access and enhancing privacy protections against both insider and external threats [116, 124–126]. Deploying containers within CVM further secures the software stack, enhancing defense through isolation, runtime encryption, sealing, and attestation. However, integrating the entire software stack within the TCB introduces complexities. The container software stack comprises multiple layers, each with inherent vulnerabilities; expanding the TCB increases the potential attack surface if all components are assumed entirely secure. To address these, our framework leverages the single-user, single-application deployment model ($\mathcal{I}_2$). This focused approach tailors each container to specific applications and users, streamlining the TCB by eliminating non-essential components and establishing inherent trust within its boundaries. Maintaining integrity within a mutable TCB necessitates robust solutions. Technologies such as TPMs [127] and Linux's IMA [115] are employed to ensure hardware-based data integrity. However, relying on TPMs involves a degree of trust in the cloud

provider, which conflicts with minimizing such dependence [128]. Additionally, the IMA framework lacks the granularity required for effective integrity verification at the container level, posing challenges for large-scale deployments [78, 125, 129]. Thus, emerging technologies are actively refining these aspects of container security. Innovations such as Library OS [104, 130, 131] and unikernels [111, 132, 133] address TPM challenges within confidential computing environments [128]. Simultaneously, advancements are being made to adapt the IMA to be more container-aware and suitable for containers [67, 77–79, 129, 134], thereby enhancing its applicability and effectiveness in ensuring integrity at scale.

7 Conclusion and Future Directions

This survey fills a gap in the literature by presenting a novel taxonomy that categorizes vulnerabilities, attacks, and mitigations across three dimensions (see Section 1). Through an in-depth analysis of over 200 container-related vulnerabilities, we identified 47 exploit types and 11 attack categories, advancing the understanding of container security from the kernel to the application level (see Section 3). Our proposed framework not only strengthens containerized systems in complex, multi-tenant cloud environments but also provides actionable insights for practitioners to enhance security.

Key findings include the identification of exploit types and attack vectors prevalent in containers, along with corresponding mitigations. This mapping aids practitioners in implementing targeted defenses. We recommend using our taxonomy to assess and mitigate risks in containerized environments. By understanding these exploit types and attack vectors, teams can prioritize resources and deploy countermeasures, enhancing resilience against threats.

Building on insights from Section 5, future research must focus on developing an integrated framework to enhance confidentiality and integrity in containers (see Section 6). Specifically, we will concentrate on:

- **Developing Deployment Models for Confidential Containers:** This involves creating fast, minimal, optimized, and scalable OSs tailored for confidential containers. We will explore lightweight OS designs that maintain performance and scalability without compromising security.
- **Advancing Unikernel Deployment:** Unikernel challenges such as supporting many containers and ensuring isolation persist. Solutions show potential but need further research to address static linking, single-application deployment, and multiprocessing [112, 113, 132].
- **Privacy-preserving Attestation Models for Containers:** Future research should focus on developing container-suited attestation models, emphasizing improved privacy and scalability

In conclusion, our study provides a foundation for both practitioners and researchers to enhance cloud container security. By leveraging our taxonomy and findings, practitioners can implement more effective strategies, while researchers can build upon our work to address emerging challenges.

References

[1] Dirk Merkel. 2014. Docker: Lightweight Linux containers for consistent development and deployment. *Linux J.* 2014, 239 (Mar. 2014), 2:2.

[2] Roberto Morabito. 2015. Hypervisors vs. lightweight virtualization: A performance comparison. In *Proceedings of the IEEE Conference on Cloud Engineering*. Retrieved from https://ieeexplore.ieee.org/document/7092949

[3] Stephen Soltesz, Herbert Pötzl, Marc E. Fiuczynski, Andy Bavier, and Larry Peterson. 2007. Container-based operating system virtualization: A scalable, high-performance alternative to hypervisors. In *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys'07)*. Association for Computing Machinery, 275–287. DOI: http://doi.org/10.1145/1272996.1273025

[4] Sari Sultan, Imtiaz Ahmad, and Tassos Dimitriou. 2019. Container security: Issues, challenges, and the road ahead. *IEEE Access* 7 (2019), 52976–52996. Retrieved from https://ieeexplore.ieee.org/abstract/document/8693491/

[5] Xing Gao, Zhongshu Gu, Mehmet Kayaalp, Dimitrios Pendarakis, and Haining Wang. 2017. ContainerLeaks: Emerging security threats of information leakages in container clouds. In *Proceedings of the 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'17)*. IEEE, 237–248. Retrieved from https://ieeexplore.ieee.org/abstract/document/8023126/

[6] Byungchul Tak, Canturk Isci, Sastry Duri, Nilton Bila, Shripad Nadgowda, and James Doran. 2017. Understanding security implications of using containers in the cloud. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'17)*. 313–319. Retrieved from https://www.usenix.org/conference/atc17/technical-sessions/presentation/tak

[7] Luis M. Vaquero, Luis Rodero-Merino, and Daniel Morán. 2011. Locking the sky: A survey on IaaS cloud security. *Computing* 91, 1 (Jan. 2011), 93–118. DOI: http://doi.org/10.1007/s00607-010-0140-x

[8] Claus Pahl. 2015. Containerization and the PaaS cloud. *IEEE Cloud Comput.* 2, 3 (2015), 24–31. Retrieved from https://ieeexplore.ieee.org/abstract/document/7158965/

[9] Maxime Bélair, Sylvie Laniepce, and Jean-Marc Menaud. 2019. Leveraging kernel security mechanisms to improve container security: A survey. In *Proceedings of the 14th International Conference on Availability, Reliability and Security*. ACM, 1–6. DOI: http://doi.org/10.1145/3339252.3340502

[10] Ali Gholami and Erwin Laure. 2015. Security and privacy of sensitive data in cloud computing: A survey of recent developments. In *Proceedings of the Computer Science & Information Technology Conference (CS & IT'15)*. 131–150. DOI: http://doi.org/10.5121/csit.2015.51611

[11] Yutian Yang, Wenbo Shen, Bonan Ruan, Wenmao Liu, and Kui Ren. 2021. Security challenges in the container cloud. In *Proceedings of the 3rd IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA'21)*. IEEE, 137–145. Retrieved from https://ieeexplore.ieee.org/abstract/document/9750244/

[12] Prashant Desai. 2016. A survey of performance comparison between virtual machines and containers. *Int. J. Comput. Sci. Eng.* 4 (July 2016), 55–59.

[13] Andrea Tosatto, Pietro Ruiu, and Antonio Attanasio. 2015. Container-based orchestration in cloud: State of the art and challenges. In *Proceedings of the 9th International Conference on Complex, Intelligent, and Software Intensive Systems*. IEEE, 70–75. Retrieved from https://ieeexplore.ieee.org/abstract/document/7185168/

[14] Junzo Watada, Arunava Roy, Ruturaj Kadikar, Hoang Pham, and Bing Xu. 2019. Emerging trends, techniques and open issues of containerization: A review. *IEEE Access* 7 (2019), 152443–152472. Retrieved from https://ieeexplore.ieee.org/abstract/document/8861307/

[15] Federico Sierra-Arriaga, Rodrigo Branco, and Ben Lee. 2021. Security issues and challenges for virtualization technologies. *Comput. Surv.* 53, 2 (Mar. 2021), 1–37. DOI: http://doi.org/10.1145/3382190

[16] Shufan Fei, Zheng Yan, Wenxiu Ding, and Haomeng Xie. 2022. Security vulnerabilities of SGX and countermeasures: A survey. *Comput. Surv.* 54, 6 (July 2022), 1–36. DOI: http://doi.org/10.1145/3456631

[17] Ann Yi Wong, Eyasu Getahun Chekole, Martin Ochoa, and Jianying Zhou. 2021. Threat modeling and security analysis of containers: A survey. DOI: http://doi.org/10.48550/arXiv.2111.11475

[18] Claus Pahl, Antonio Brogi, Jacopo Soldani, and Pooyan Jamshidi. 2017. Cloud container technologies: A state-of-the-art review. *IEEE Trans. Cloud Comput.* 7, 3 (2017), 677–692. Retrieved from https://ieeexplore.ieee.org/abstract/document/7922500/

[19] Xin Lin, Lingguang Lei, Yuewu Wang, Jiwu Jing, Kun Sun, and Quan Zhou. 2018. A measurement study on Linux container security: Attacks and countermeasures. In *Proceedings of the 34th Annual Computer Security Applications Conference*. ACM, 418–429. DOI: http://doi.org/10.1145/3274694.3274720

[20] NIST. 2024. National vulnerability dataset. Retrieved from https://nvd.nist.gov/

[21] MITRE. 2024. MITRE ATT&CK®. Retrieved February 4, 2025 from https://attack.mitre.org/

[22] Michael Pearce, Sherali Zeadally, and Ray Hunt. 2013. Virtualization: Issues, security threats, and solutions. *Comput. Surv.* 45, 2 (Feb. 2013), 1–39. DOI: http://doi.org/10.1145/2431211.2431216

[23] Ken Barr, Prashanth Bungale, Stephen Deasy, Viktor Gyuris, Perry Hung, Craig Newell, Harvey Tuch, and Bruno Zoppis. 2010. The VMware mobile virtualization platform: Is that a hypervisor in your pocket? *ACM SIGOPS Operat. Syst. Rev.* 44, 4 (Dec. 2010), 124–135. DOI: http://doi.org/10.1145/1899928.1899945

[24] Edward Haletky. 2011. *VMware ESX and ESXi in the Enterprise: Planning Deployment of Virtualization Servers*. Pearson Education. Retrieved from https://books.google.com.au/books?hl=en&lr=&id=cysH9aWVdiAC&oi=fnd&pg=PT28&dq=vmware+esxi+{\}documentation&ots=rEBY9i81rv&sig=GIhVCcQ7vz5GXPYvqSGPZUyGdfs

[25] Oracle Inc. 2011. 1.1.1. Brief history of virtualization. Retrieved from https://docs.oracle.com/cd/E26996_01/E18549/html/VMUSG1010.html

[26] David Rensin. 2015. *Kubernetes*. O'Reilly Media, Incorporated. Retrieved from https://cloud.redhat.com/hubfs/pdfs/Kubernetes_OpenShift.pdf

[27] T. Kubernetes. 2019. Kubernetes. *Kubernetes*. Retrieved from https://dzone.com/storage/attachments/14131598-dzone-kubernetes-bundle.pdf
[28] Nikhil Marathe, Ankita Gandhi, and Jaimeel M. Shah. 2019. Docker swarm and Kubernetes in cloud computing environment. In *Proceedings of the 3rd International Conference on Trends in Electronics and Informatics (ICOEI'19)*. IEEE, 179–184. Retrieved from https://ieeexplore.ieee.org/abstract/document/8862654/
[29] Pankaj Saha, Madhusudhan Govindaraju, Suresh Marru, and Marlon Pierce. 2016. Integrating Apache Airavata with Docker, Marathon, and Mesos. *Concurr. Computat.: Pract. Exper.* 28, 7 (May 2016), 1952–1959. DOI: http://doi.org/10.1002/cpe.3708
[30] Emiliano Casalicchio and Stefano Iannucci. 2020. The state-of-the-art in container technologies: Application, orchestration and security. *Concurr. Computat.: Pract. Exper.* 32, 17 (Sept. 2020), e5668. DOI: http://doi.org/10.1002/cpe.5668
[31] Fabrizio Soppelsa and Chanwit Kaewkasi. 2016. *Native Docker Clustering with Swarm*. Packt Publishing Ltd. Retrieved from https://books.google.com.au/books?hl=en&lr=&id=Xc_cDgAAQBAJ&oi=fnd&pg=PP1&dq=docker+swarm+{\}documentation&ots=U2UKUuREQJ&sig=BPB8xW206xTJswO5U8GVwtmjl98
[32] Lennart Espe, Anshul Jindal, Vladimir Podolskiy, and Michael Gerndt. 2020. Performance evaluation of container runtimes. In *Proceedings of the 10th International Conference on Cloud Computing (CLOSER'20)*. 273–281. Retrieved from https://pdfs.semanticscholar.org/af89/2db0220dc3e48b953070a6f8573349d8490d.pdf
[33] Antony Martin, Simone Raponi, Théo Combe, and Roberto Di Pietro. 2018. Docker ecosystem–vulnerability analysis. *Comput. Commun.* 122 (2018), 30–43. Retrieved from https://www.sciencedirect.com/science/article/pii/S0140366417300956
[34] opencontainers. 2024. opencontainers/runc. Retrieved February 4, 2025 from https://github.com/opencontainers/runc
[35] containerd. 2024. containerd/containerd. Retrieved February 4, 2025 from https://github.com/containerd/containerd
[36] Scrivano Giuseppe. 2024. containers/crun. Retrieved from https://github.com/containers/crun
[37] Eddy Truyen, Dimitri Van Landuyt, Vincent Reniers, Ansar Rafique, Bert Lagaisse, and Wouter Joosen. 2016. Towards a container-based architecture for multi-tenant SaaS applications. In *Proceedings of the 15th International Workshop on Adaptive and Reflective Middleware*. ACM, 1–6. DOI: http://doi.org/10.1145/3008167.3008173
[38] Nanzi Yang, Wenbo Shen, Jinku Li, Yutian Yang, Kangjie Lu, Jietao Xiao, Tianyu Zhou, Chenggang Qin, Wang Yu, Jianfeng Ma, and Kui Ren. 2021. Demons in the shared kernel: Abstract resource attacks against OS-level virtualization. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 764–778. DOI: http://doi.org/10.1145/3460120.3484744
[39] Ross McIlroy, Jaroslav Sevcik, Tobias Tebbi, Ben L. Titzer, and Toon Verwaest. 2019. Spectre is here to stay: An analysis of side-channels and speculative execution. Retrieved from http://arxiv.org/abs/1902.05178
[40] Ofek Kirzner and Adam Morrison. 2021. An analysis of speculative type confusion vulnerabilities in the wild. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security'21)*. 2399–2416. Retrieved from https://www.usenix.org/conference/usenixsecurity21/presentation/kirzner
[41] Trend Micro. 2019. Kubernetes vulnerability CVE-2019-11246 discovered due to incomplete updates from a previous flaw - Security news. Retrieved February 4, 2025 from https://www.trendmicro.com/vinfo/au/security/news/vulnerabilities-and-exploits/kubernetes-vulnerability-cve-2019-11246-discovered-due-to-incomplete-updates-from-a-previous-flaw
[42] The Kubernetes Project. 2020. CVE-2020-8552: Apiserver DoS (oom) · Issue #89378 · kubernetes/kubernetes. Retrieved February 4, 2025 from https://github.com/kubernetes/kubernetes/issues/89378
[43] Snyk. 2023. Snyk vulnerability database. Retrieved February 4, 2025 from https://security.snyk.io/
[44] GitHub Advisory Database. 2023. CVE-2023-3978 - GitHub advisory database. Retrieved February 4, 2025 from https://github.com/advisories/GHSA-2wrh-6pvc-2jm9
[45] Eric Rescorla. 2018. *The Transport Layer Security (TLS) Protocol Version 1.3*. Request for Comments RFC 8446. Internet Engineering Task Force. DOI: http://doi.org/10.17487/RFC8446
[46] Kubernetes. 2020. [Security Advisory] CVE-2019-11254: Denial of service vulnerability from malicious YAML payloads - Announcements. Retrieved February 4, 2025 from https://discuss.kubernetes.io/t/security-advisory-cve-2019-11254-denial-of-service-vulnerability-from-malicious-yaml-payloads/10345
[47] Kubernetes. 2019. CVE-2019-11253: Kubernetes API Server JSON/YAML parsing vulnerable to resource exhaustion attack. Issue #83253. kubernetes/kubernetes. Retrieved February 4, 2025 from https://github.com/kubernetes/kubernetes/issues/83253
[48] kubernetes-client. 2021. CVE-2021-25738: Code exec via YAML parsing. Issue #1698. kubernetes-client/java. Retrieved February 4, 2025 from https://github.com/kubernetes-client/java/issues/1698
[49] bgeesaman. 2017. bgeesaman/subpath-exploit: Writeup of CVE-2017-1002101 with sample "exploit"/escape. Retrieved February 4, 2025 from https://github.com/bgeesaman/subpath-exploit

[50] Moby. 2024. moby/moby. Retrieved February 4, 2025 from https://github.com/moby/moby
[51] Kun Suo, Yong Zhao, Wei Chen, and Jia Rao. 2018. An analysis and empirical study of container networks. In *Proceedings of the IEEE Conference on Computer Communications (INFOCOM'18)*. IEEE, 189–197. Retrieved from https://ieeexplore.ieee.org/abstract/document/8485865/
[52] Haitham Ameen Noman and Osama M. F. Abu-Sharkh. 2023. Code injection attacks in wireless-based Internet of Things (IoT): A comprehensive review and practical implementations. *Sensors* 23, 13 (2023), 6067. Retrieved from https://www.mdpi.com/1424-8220/23/13/6067
[53] Trend Micro. 2014. Protect against Shellshock Linux Bash Vulnerability - OfficeScan. Retrieved February 4, 2025 from https://success.trendmicro.com/dcx/s/solution/1105238-officescan-protection-against-shellshock-linux-bash-vulnerability
[54] Alessandro Randazzo and Ilenia Tinnirello. 2019. Kata containers: An emerging architecture for enabling MEC services in fast and secure way. In *Proceedings of the 6th International Conference on Internet of Things: Systems, Management and Security (IOTSMS'19)*. IEEE, 209–214. Retrieved from https://ieeexplore.ieee.org/abstract/document/8939164/
[55] Google. [n.d.]. SELinux. Retrieved February 4, 2025 from https://www.google.com/search?q=selinux
[56] AppArmor. [n.d.]. AppArmor. Retrieved February 4, 2025 from https://apparmor.net/
[57] Zhiqiang Jian and Long Chen. 2017. A defense method against Docker escape attack. In *Proceedings of the International Conference on Cryptography, Security and Privacy*. ACM, 142–146. DOI: http://doi.org/10.1145/3058060.3058085
[58] Xing Gao, Zhongshu Gu, Zhengfa Li, Hani Jamjoom, and Cong Wang. 2019. Houdini's escape: Breaking the resource rein of Linux control groups. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1073–1086. DOI: http://doi.org/10.1145/3319535.3354227
[59] Thu Yein Win, Fung Po Tso, Quentin Mair, and Huaglory Tianfield. 2017. Protect: Container process isolation using system call interception. In *Proceedings of the 14th International Symposium on Pervasive Systems, Algorithms and Networks, 11th International Conference on Frontier of Computer Science and Technology, 3rd International Symposium of Creative Computing (ISPAN-FCST-ISCC'17)*. IEEE, 191–196. Retrieved from https://ieeexplore.ieee.org/abstract/document/8121772/
[60] Debadatta Mishra and Purushottam Kulkarni. 2018. A survey of memory management techniques in virtualized systems. *Comput. Sci. Rev.* 29 (2018), 56–73. Retrieved from https://www.sciencedirect.com/science/article/pii/S1574013716301186
[61] Junaid Khalid, Eric Rozner, Wesley Felter, Cong Xu, Karthick Rajamani, Alexandre Ferreira, and Aditya Akella. 2018. Iron: Isolating network-based CPU} in container environments. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI'18)*. 313–328. Retrieved from https://www.usenix.org/conference/nsdi18/presentation/khalid
[62] Andreas Haeberlen and Kevin Elphinstone. 2003. User-level management of kernel memory. In *Advances in Computer Systems Architecture*, Gerhard Goos, Juris Hartmanis, Jan Van Leeuwen, Amos Omondi, and Stanislav Sedukhin (Eds.). *(Lecture Notes in Computer Science,* Vol. 2823). Springer Berlin, 277–289. DOI: http://doi.org/10.1007/978-3-540-39864-6_23
[63] Victor Marmol, Rohit Jnagal, and Tim Hockin. 2015. Networking in containers and container clusters. In *Proceedings of Netdev 0.1*. 14–17. Retrieved from https://netdevconf.info/0.1/papers/Networking-in-Containers-and-Container-Clusters.pdf
[64] Sebastiano Miano, Fulvio Risso, Mauricio Vásquez Bernal, Matteo Bertrone, and Yunsong Lu. 2021. A framework for eBPF-based network functions in an era of microservices. *IEEE Trans. Netw. Serv. Manag.* 18, 1 (2021), 133–151. Retrieved from https://ieeexplore.ieee.org/abstract/document/9340283/ Publisher: IEEE.
[65] Michael Rash. 2007. *Linux Firewalls: Attack Detection and Response*. No Starch Press. Retrieved from https://books.google.com.au/books?hl=en&lr=&id=Ad9Fnk9C7QsC&oi=fnd&pg=PR15&dq=linux+firewall+attacks+{\}detection+response&ots=KT8iyQT4Rq&sig=YxIaKw6RRJr2MrvsuhenRv0u2EQ
[66] Jaehyun Nam, Seungsoo Lee, Hyunmin Seo, Phil Porras, Vinod Yegneswaran, and Seungwon Shin. 2020. BASTION: A security enforcement network stack for container networks. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'20)*. 81–95. Retrieved from https://www.usenix.org/conference/atc20/presentation/nam
[67] Yuqiong Sun, David Safford, Mimi Zohar, Dimitrios Pendarakis, Zhongshu Gu, and Trent Jaeger. 2018. Security namespace: Making Linux security frameworks available to containers. In *Proceedings of the 27th USENIX Security Symposium (USENIX Security'18)*. 1423–1439. Retrieved from https://www.usenix.org/conference/usenixsecurity18/presentation/sun
[68] Yi He, Roland Guo, Yunlong Xing, Xijia Che, Kun Sun, Zhuotao Liu, Ke Xu, and Qi Li. 2023. Cross container attacks: The bewildered eBPF on clouds. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security'23)*. 5971–5988. Retrieved from https://www.usenix.org/conference/usenixsecurity23/presentation/he

[69] Ralf Hund, Carsten Willems, and Thorsten Holz. 2013. Practical timing side channel attacks against kernel space ASLR. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'13)*. IEEE, 191–205. Retrieved from https://ieeexplore.ieee.org/abstract/document/6547110/

[70] Dave (Jing) Tian, Joseph I. Choi, Grant Hernandez, Patrick Traynor, and Kevin R. B. Butler. 2019. A practical Intel SGX setting for Linux containers in the cloud. In *Proceedings of the 9th ACM Conference on Data and Application Security and Privacy*. ACM, 255–266. DOI: http://doi.org/10.1145/3292006.3300030

[71] Antonio Barresi, Kaveh Razavi, Mathias Payer, and Thomas R. Gross. 2015. CAIN: Silently breaking ASLR in the cloud. In *Proceedings of the 9th USENIX Workshop on Offensive Technologies (WOOT'15)*. Retrieved from https://www.usenix.org/conference/woot15/workshop-program/presentation/barresi

[72] Yeongjin Jang, Sangho Lee, and Taesoo Kim. 2016. Breaking kernel address space layout randomization with intel TSX. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 380–392. DOI: http://doi.org/10.1145/2976749.2978321

[73] Dmitry Evtyushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. 2016. Jump over ASLR: Attacking branch predictors to bypass ASLR. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'16)*. IEEE, 1–13. Retrieved from https://ieeexplore.ieee.org/abstract/document/7783743/

[74] Daniel Gruss, Clémentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. 2016. Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 368–379. DOI: http://doi.org/10.1145/2976749.2978356

[75] Ben Gras, Kaveh Razavi, Erik Bosman, Herbert Bos, and Cristiano Giuffrida. 2017. ASLR on the line: Practical cache attacks on the MMU. In *Proceedings of the NDSS Symposium (NDSS'17)*. 26. Retrieved from http://hydra.azilian.net/Papers/Security/Practical%20Cache%20Attacks%20on%20the%20MMU.pdf

[76] Johannes Wikner and Kaveh Razavi. 2022. RETBLEED: Arbitrary speculative code execution with return instructions. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security'22)*. 3825–3842. Retrieved from https://www.usenix.org/conference/usenixsecurity22/presentation/wikner

[77] J. Corbet (LWN.net). 2020. IMA: Introduce IMA namespace. Retrieved February 4, 2025 from https://lwn.net/Articles/829035/

[78] Asier Gutierrez. 2023. IMA namespaces for containers. Retrieved from https://www.youtube.com/watch?v=bs7EcuF4SSo&t=8s&ab_channel=TheLinuxFoundation

[79] Wu Luo, Qingni Shen, Yutang Xia, and Zhonghai Wu. 2019. Container-IMA: A privacy-preserving integrity measurement architecture for containers. In *Proceedings of the 22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID'19)*. 487–500. Retrieved from https://www.usenix.org/conference/raid2019/presentation/luo

[80] The Linux Foundation. 2022. IMA policy support for fs-verity: A win-win for IMA & fs-verity - Mimi Zohar, IBM. Retrieved from https://www.youtube.com/watch?v=7GMA6VUcsdM

[81] Luke Hinds. 2020. [RFC PATCH 00/30] IMA: Introduce IMA namespace. Retrieved from https://lists.linuxfoundation.org/pipermail/containers/2020-September/042127.html

[82] Mimi Zohar. 2020. [RFC,00/30] IMA: Introduce IMA namespace - Patchwork. Retrieved from https://patchwork.kernel.org/project/linux-integrity/cover/20200818152037.11869-1-krzysztof.struczynski@huawei.com/#23587721

[83] Chit-Kwan Lin and Stephen J. Tarsa. 2019. Branch prediction is not a solved problem: Measurements, opportunities, and future directions. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC'19)*. 228–238. DOI: http://doi.org/10.1109/IISWC47752.2019.9042108

[84] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, et al. 2020. Meltdown: Reading kernel memory from user space. *Commun. ACM* 63, 6 (May 2020), 46–56. DOI: http://doi.org/10.1145/3357033

[85] Chao Li, Zhenhua Wang, Xiaofeng Hou, Haopeng Chen, Xiaoyao Liang, and Minyi Guo. 2016. Power attack defense: Securing battery-backed data centers. *ACM SIGARCH Comput. Archit. News* 44, 3 (Oct. 2016), 493–505. DOI: http://doi.org/10.1145/3007787.3001189

[86] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher et al.. 2020. Spectre attacks: Exploiting speculative execution. *Commun. ACM* 63, 7 (June 2020), 93–101. DOI: http://doi.org/10.1145/3399742

[87] Chaoqun Shen, Congcong Chen, and Jiliang Zhang. 2021. Micro-architectural cache side-channel attacks and countermeasures. In *Proceedings of the 26th Asia and South Pacific Design Automation Conference*. ACM, 441–448. DOI: http://doi.org/10.1145/3394885.3431638

[88] Yueqiang Cheng, Zhi Zhang, Yansong Gao, Zhaofeng Chen, Shengjian Guo, Qifei Zhang, Rui Mei, Surya Nepal, and Yang Xiang. 2022. Meltdown-type attacks are still feasible in the wall of kernel page-Table isolation. *Comput. Secur.* 113 (2022), 102556. Retrieved from https://www.sciencedirect.com/science/article/pii/S0167404821003801

[89] Kaspersky. 2023. Zenbleed: Hardware vulnerability in AMD CPUs. Kaspersky official blog. Retrieved February 4, 2025 from https://www.kaspersky.com.au/blog/zenbleed-vulnerability/32394/

[90] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. 2019. ZombieLoad: Cross-privilege-boundary data sampling. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 753–768. DOI: http://doi.org/10.1145/3319535.3354252

[91] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. 2018. Foreshadow: Extracting the keys to the Intel SGX kingdom with transient Out-of-Order execution. In *Proceedings of the 27th USENIX Security Symposium (USENIX Security'18)*. 991–1008. Retrieved from https://www.usenix.org/conference/usenixsecurity18/presentation/bulck

[92] Kit Murdock, David Oswald, Flavio D. Garcia, Jo Van Bulck, Daniel Gruss, and Frank Piessens. 2020. Plundervolt: Software-based fault injection attacks against Intel SGX. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'20)*. IEEE, 1466–1482. Retrieved from https://ieeexplore.ieee.org/abstract/document/9152636/

[93] Ferdinand Brasser, Urs Müller, Alexandra Dmitrienko, Kari Kostiainen, Srdjan Capkun, and Ahmad-Reza Sadeghi. 2017. Software grand exposure: SGX cache attacks are practical. In *Proceedings of the 11th USENIX Workshop on Offensive Technologies (WOOT'17)*. Retrieved from https://www.usenix.org/conference/woot17/workshop-program/presentation/brasser

[94] Stephan Van Schaik, Marina Minkin, Andrew Kwong, Daniel Genkin, and Yuval Yarom. 2021. CacheOut: Leaking data on Intel CPUs via cache evictions. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'21)*. IEEE, 339–354. Retrieved from https://ieeexplore.ieee.org/abstract/document/9519461/

[95] Marina Minkin, Daniel Moghimi, Moritz Lipp, Michael Schwarz, Jo Van Bulck, Daniel Genkin, Daniel Gruss, Frank Piessens, Berk Sunar, and Yuval Yarom. 2019. Fallout: Reading kernel writes from user space. Retrieved from http://arxiv.org/abs/1905.12701

[96] Stephan Van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2019. RIDL: Rogue in-flight data load. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'19)*. IEEE, 88–105. Retrieved from https://ieeexplore.ieee.org/abstract/document/8835281/

[97] Moritz Lipp, Andreas Kogler, David Oswald, Michael Schwarz, Catherine Easdon, Claudio Canella, and Daniel Gruss. 2021. PLATYPUS: Software-based power side-channel attacks on x86. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'21)*. IEEE, 355–371. Retrieved from https://ieeexplore.ieee.org/abstract/document/9519416/

[98] OAuth.net. 2012. OAuth 2.0 – OAuth. Retrieved February 4, 2025 from https://oauth.net/2/

[99] Dazan Qian, Songhui Guo, Lei Sun, Haidong Liu, Qianfang Hao, and Jing Zhang. 2020. Trusted virtual network function based on vTPM. In *Proceedings of the 7th International Conference on Information Science and Control Engineering (ICISCE'20)*. IEEE, 1484–1488. Retrieved from https://ieeexplore.ieee.org/abstract/document/9532194/

[100] Jim Alves-Foss, Carol Taylor, and Paul Oman. 2004. A multi-layered approach to security in high assurance systems. In *Proceedings of the 37th Annual Hawaii International Conference on System Sciences (HICSS'04)*. IEEE, 10. Retrieved from https://ieeexplore.ieee.org/abstract/document/1265709/

[101] Victor Costan. 2016. Intel SGX explained. *IACR Cryptol, EPrint Arch.* Retrieved from https://people.cs.rutgers.edu/~santosh.nagarakatte/cs544/readings/costan-sgx.pdf

[102] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. 2024. Intel TDX demystified: A top-down approach. *ACM Comput. Surv.* 56, 9 (Apr. 2024), 238:1–238:33. DOI: http://doi.org/10.1145/3652597

[103] Zhichao Hua, Yang Yu, Jinyu Gu, Yubin Xia, Haibo Chen, and Binyu Zang. 2021. TZ-Container: Protecting container from untrusted OS with ARM TrustZone. *Sci. China Inf. Sci.* 64, 9 (Sept. 2021), 192101. DOI: http://doi.org/10.1007/s11432-019-2707-6

[104] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, Andre Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O'Keeffe, and Mark L. Stillwell. 2016. SCONE: Secure Linux containers with Intel SGX. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI'16)*. 689–703. Retrieved from https://www.usenix.org/conference/osdi16/technical-sessions/presentation/arnautov

[105] deislabs. 2024. deislabs/mystikos. (Apr. 2024). Retrieved February 4, 2025 from https://github.com/deislabs/mystikos

[106] David Kaplan, Jeremy Powell, and Tom Woller. 2016. AMD memory encryption. *White Pap.* 13 (2016). Retrieved from https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/memory-encryption-white-paper.pdf

[107] Shixuan Zhao, Mengyuan Li, Yinqian Zhangyz, and Zhiqiang Lin. 2022. vSGX: Virtualizing SGX enclaves on AMD SEV. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'22)*. IEEE, 321–336. Retrieved from https://ieeexplore.ieee.org/abstract/document/9833694/

[108] Arch Linux. [n.d.]. Kata Containers - ArchWiki. Retrieved February 4, 2025 from https://wiki.archlinux.org/title/Kata_Containers

[109] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight virtualization for serverless applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI'20)*. 419–434. Retrieved from https://www.usenix.org/conference/nsdi20/presentation/agache

[110] Alexander Van't Hof and Jason Nieh. 2022. BlackBox: A container security monitor for protecting containers on untrusted operating systems. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI'22)*. 683–700. Retrieved from https://www.usenix.org/conference/osdi22/presentation/vant-hof

[111] Hsuan-Chi Kuo, Dan Williams, Ricardo Koller, and Sibin Mohan. 2020. A Linux in unikernel clothing. In *Proceedings of the 15th European Conference on Computer Systems*. ACM, 1–15. DOI: http://doi.org/10.1145/3342195.3387526

[112] Ali Raza, Thomas Unger, Matthew Boyd, Eric B. Munson, Parul Sohal, Ulrich Drepper, Richard Jones, Daniel Bristot De Oliveira, Larry Woodman, Renato Mancuso et al. 2023. Unikernel Linux (UKL). In *Proceedings of the 18th European Conference on Computer Systems*. ACM, 590–605. DOI: http://doi.org/10.1145/3552326.3587458

[113] Ali Raza, Parul Sohal, James Cadden, Jonathan Appavoo, Ulrich Drepper, Richard Jones, Orran Krieger, Renato Mancuso, and Larry Woodman. 2019. Unikernels: The next stage of Linux's dominance. In *Proceedings of the Workshop on Hot Topics in Operating Systems*. ACM, 7–13. DOI: http://doi.org/10.1145/3317550.3321445

[114] Christian Brauner. 2020. Seccomp Notify - New frontiers in unprivileged container development. Retrieved from https://people.kernel.org/brauner/the-seccomp-notifier-new-frontiers-in-unprivileged-container-development

[115] Reiner Sailer, Xiaolan Zhang, Trent Jaeger, and Leendert van Doorn. 2004. Design and implementation of a TCG-based integrity measurement architecture. Retrieved from https://www.usenix.org/conference/13th-usenix-security-symposium/design-and-implementation-tcg-based-integrity-measurement

[116] Edgeless Systems. 2021. Confidential computing 101 by Felix Schuster (edgeless systems). OC3 2021. Retrieved from https://www.youtube.com/watch?v=77U12Ss38Zc

[117] Linux Foundation Project. 2024. Confidential computing consortium. Retrieved from https://confidentialcomputing.io/

[118] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. 2015. Trusted execution environment: What it is, and what it is not. In *Proceedings of the IEEE Trustcom/BigDataSE/ISPA Conference*. IEEE, 57–64. Retrieved from https://ieeexplore.ieee.org/abstract/document/7345265/

[119] N. de Bruijn. 2017. eBPF based networking. Retrieved from https://rp.os3.nl/2016-2017/p84/report.pdf

[120] Gerald Budigiri, Christoph Baumann, Jan Tobias Mühlberg, Eddy Truyen, and Wouter Joosen. 2021. Network policies in Kubernetes: Performance evaluation and security analysis. In *Proceedings of the Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit'21)*. IEEE, 407–412. Retrieved from https://ieeexplore.ieee.org/abstract/document/9482526/

[121] Datadog. [n.d.]. Cloud Monitoring as a Service. Datadog. Retrieved February 4, 2025 from https://www.datadoghq.com/

[122] Cilium. 2024. cilium/hubble. Retrieved February 4, 2025 from https://github.com/cilium/hubble

[123] Filipe Manco, Costin Lupu, Florian Schmidt, Jose Mendes, Simon Kuenzer, Sumit Sati, Kenichi Yasukata, Costin Raiciu, and Felipe Huici. 2017. My VM is lighter (and safer) than your container. In *Proceedings of the 26th Symposium on Operating Systems Principles*. ACM, 218–233. DOI: http://doi.org/10.1145/3132747.3132763

[124] Muhammad Usama Sardar and Christof Fetzer. 2023. Confidential computing and related technologies: A critical review. *Cybersecurity* 6, 1 (May 2023), 10. DOI: http://doi.org/10.1186/s42400-023-00144-1

[125] Muhammad Usama Sardar, Arto Niemi, Hannes Tschofenig, and Thomas Fossati. 2024. Towards validation of TLS 1.3 formal model and vulnerabilities in Intel's RA-TLS protocol. *IEEE Access*. https://www.researchgate.net/publication/385384309_Towards_Validation_of_TLS_13_Formal_Model_and_Vulnerabilities_in_Intel's_RA-TLS_Protocol

[126] Muhammad Usama Sardar, Saidgani Musaev, and Christof Fetzer. 2021. Demystifying attestation in Intel trust domain extensions via formal verification. *IEEE Access* 9 (2021), 83067–83079. Retrieved from https://ieeexplore.ieee.org/abstract/document/9448036/

[127] Ronald Perez, Reiner Sailer, and Leendert van Doorn. 2006. vTPM: Virtualizing the trusted platform module. In *Proceedings of the 15th USENIX Security Symposium*. 305–320. Retrieved from https://www.usenix.org/event/sec06/tech/full_papers/berger/berger_html/

[128] Vikram Narayanan, Claudio Carvalho, Angelo Ruocco, Gheorghe Almasi, James Bottomley, Mengmei Ye, Tobin Feldman-Fitzthum, Daniele Buono, Hubertus Franke, and Anton Burtsev. 2023. Remote attestation of confidential VMs using ephemeral vTPMs. In *Proceedings of the Annual Computer Security Applications Conference*. ACM, 732–743. DOI: http://doi.org/10.1145/3627106.3627112

[129] Red Hat Community. 2023. Container IMA using eBPF - Container Plumbing Days 2023. Retrieved from https://www.youtube.com/watch?v=PnhL8bGdBz4

[130] Gramine Project. [n.d.]. Gramine. Retrieved February 4, 2025 from https://gramineproject.io/

[131] Christian Priebe, Divya Muthukumaran, Joshua Lind, Huanzhou Zhu, Shujie Cui, Vasily A. Sartakov, and Peter Pietzuch. 2020. SGX-LKL: Securing the host OS interface for trusted execution. Retrieved from http://arxiv.org/abs/1908.11143

[132] unikernelLinux. 2024. unikernelLinux/ukl. Retrieved February 4, 2025 from https://github.com/unikernelLinux/ukl

[133] Zhiming Shen, Zhen Sun, Gur-Eyal Sela, Eugene Bagdasaryan, Christina Delimitrou, Robbert Van Renesse, and Hakim Weatherspoon. 2019. X-Containers: Breaking down barriers to improve performance and isolation of cloud-native containers. In *Proceedings of the24th International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 121–135. DOI: http://doi.org/10.1145/3297858.3304016

[134] Marco De Benedictis and Antonio Lioy. 2019. Integrity verification of Docker containers for a lightweight cloud environment. *Fut. Gen. Comput. Syst.* 97 (Aug. 2019), 236–246. DOI: http://doi.org/10.1016/j.future.2019.02.026

Received 10 March 2024; revised 8 December 2024; accepted 15 January 2025